

MANUEL D'UTILISATION

Asterism

& AI Agents Map

Tes agents IA, modèles locaux, serveurs MCP et skills sur une seule carte

Asterism 0.11.0 · AI Agents Map 0.24.0

octobre 2026

À propos de ce manuel

Ce manuel décrit une application qui existe en deux éditions :

Édition	Asterism	AI Agents Map
Ce que c'est	Une app autonome GTK 4 / libadwaita pour tout bureau Linux	Un plugin pour le bureau Omarchy, ouvert depuis un bouton de la barre
S'installe depuis	AUR, AppImage ou le code source	L'Omarchy Plugin Marketplace
Apparence	Suit le mode clair/sombre et la couleur d'accent du système	Les couleurs du thème Omarchy ou l'apparence Adwaita de GNOME
Dossier des réglages	`~/.config/asterism/`	`~/.config/omarchy/ai-agents-map/`

Tout le reste — Configuration, la carte, connecter en dessinant, les LLM locaux, les modèles, les serveurs MCP et les skills — est identique dans les deux. Ce qui ne vaut que pour une édition porte la marque `ASTERISM` ou `OMARCHY`.

Conventions

Ajouter	Un bouton, un onglet ou une entrée de menu de la fenêtre.
Réglages › **Apparence**	Un chemin dans l'interface : ouvre les réglages, puis **Apparence**.
`~/.agents/skills`	Un fichier, un dossier ou une commande. `~` est ton dossier personnel.
`Ctrl`+`,`	Un raccourci clavier.

> **REMARQUE** Les remarques donnent un contexte utile.

> **ASTUCE** Les astuces montrent un chemin plus rapide ou une bonne habitude.

> **ATTENTION** Les encadrés Attention signalent quelque chose de difficile à annuler.

Sommaire

1 **Introduction** 1
- 1.1 Ce qu'il fait 1
- 1.2 Notions clés 2
- 1.3 Rien ne change sans aperçu 2

2 **Installation** 3
- 2.1 Asterism 3
- 2.2 AI Agents Map sur Omarchy 3
- 2.3 Premier démarrage 3
- 2.4 Retirer l'app 4

3 **La fenêtre en un coup d'œil** 5
- 3.1 Configuration 6
- 3.2 Choisir ce que montre la carte 8
- 3.3 Travailler avec la carte 8
- 3.4 Problèmes ouverts 10
- 3.5 Mises à jour et suggestions 10

4 **Agents** 11
- 4.1 Les détails d'un agent 11
- 4.2 Démarrer et arrêter des agents 12
- 4.3 Trafic réseau 13
- 4.4 Sous-agents 13
- 4.5 Configurer… 14
- 4.6 Se connecter 14
- 4.7 Mettre à jour des agents 15
- 4.8 Agents que l'app peut installer 15
- 4.9 Nouveaux agents 16

5 **LLM locaux** 17
- 5.1 Installer un LLM local 17
- 5.2 Démarrer et arrêter avec les agents 17
- 5.3 Les détails d'un LLM local 18
- 5.4 Mémoire GPU et libérer la mémoire GPU 18
- 5.5 Modèles en même temps 18
- 5.6 Utilisé par d'autres apps 19

6 **Modèles** 20
- 6.1 Récupérer depuis Hugging Face 20
- 6.2 Demander à un agent quel modèle convient 21
- 6.3 Installés 21
- 6.4 Taille de contexte 21
- 6.5 Sur ce PC 22
- 6.6 Déplacer et supprimer des modèles 22
- 6.7 Un seul dossier des modèles 22

7 **Connecter en dessinant** 23
- 7.1 Agent ↔ LLM local 23
- 7.2 Agent ↔ serveur MCP 23
- 7.3 Agent ↔ dossier de skills 23
- 7.4 Quand une connexion échoue 23

8 **Serveurs MCP** 24
- 8.1 D'où viennent les serveurs 24
- 8.2 Configurer un serveur 24
- 8.3 Gérer les serveurs 25

9 **Skills** 26
- 9.1 Trouver des skills 26
- 9.2 Dossiers de skills 26

10 **Réglages et aide** 27
- 10.1 Réglages 27
- 10.2 Aide 28
- 10.3 Espace disque 29

11 **Smart Check** 30
- 11.1 Résumé de la configuration du PC 31
- 11.2 Mises à jour trouvées 31
- 11.3 Statistiques de jetons 31

12 **Bilan de santé et de sécurité** 33
- 12.1 Lire les cartes 33
- 12.2 Modifier un réglage 34
- 12.3 Ce qui est vérifié pour chaque agent 34
- 12.4 Maîtriser le risque 35
- 12.5 Les mots expliqués 36

13 **Modules** 38
- 13.1 Configurer Asterism 38
- 13.2 La page des modules 39
- 13.3 Installer, activer et désactiver, mettre à jour, retirer 40
- 13.4 Niveaux de confidentialité 40
- 13.5 Types de modules 40
- 13.6 Un nouvel agent sur ton ordinateur 41
- 13.7 Créer un module 41

14 **Confidentialité et sécurité** 42

15 **Dépannage** 43

A **Référence** 44
- A.1 Raccourcis clavier 44
- A.2 Fichiers et dossiers 44
- A.3 Glossaire 45

CHAPITRE 1

Introduction

Asterism montre la configuration IA de ton ordinateur sous forme de cartes et d'une carte : quels agents de programmation sont installés, quels modèles de langage ils utilisent — services cloud et moteurs locaux —, quels modèles ceux-ci servent et quels serveurs MCP et skills chaque agent possède. Et il te permet de modifier cette configuration en dessinant des lignes.

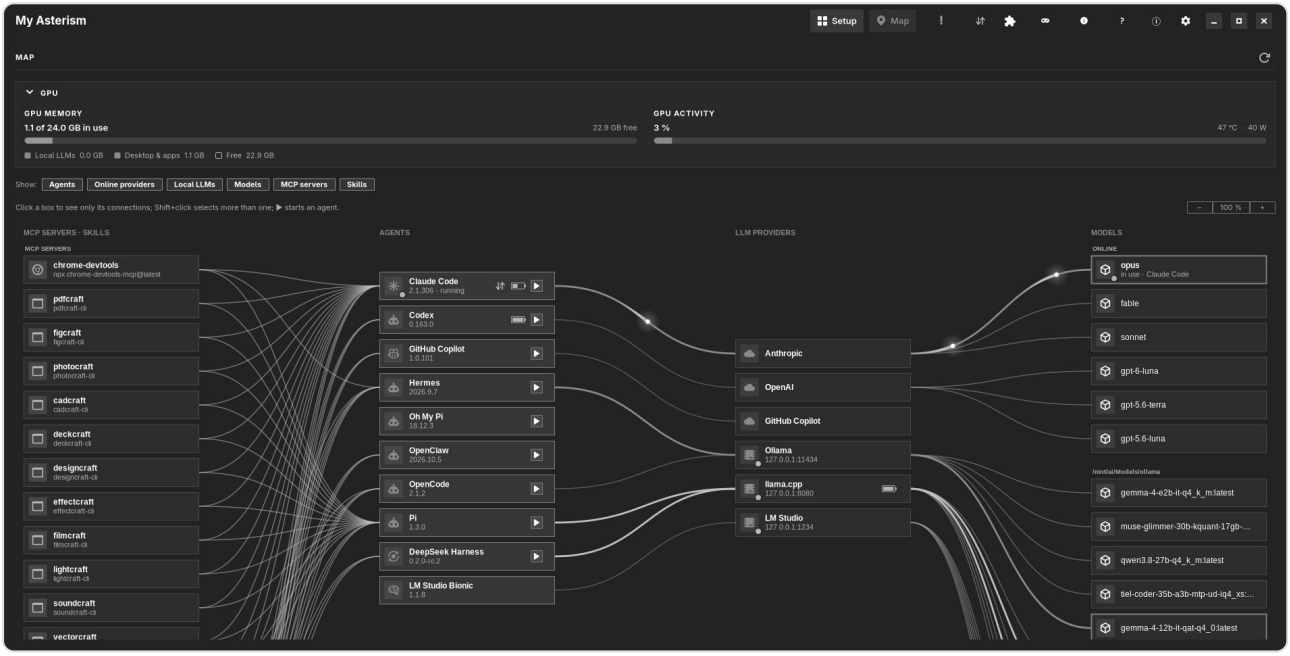


Figure 1 · La fenêtre principale : la carte sur toute la largeur, au-dessus la carte GPU, dans l'en-tête le ! des problèmes ouverts.

1.1 Ce qu'il fait

- **Agents de programmation** — Claude Code, Codex, GitHub Copilot, Cursor CLI, Gemini, OpenCode, Pi, Hermes et d'autres : leur version, leur abonnement, leurs limites et leur utilisation, et ce qui manque à chacun, avec la commande qui le corrige.
- **LLM locaux** — llama.cpp, Ollama, LM Studio, vLLM, Jan et KoboldCpp : les installer, télécharger des modèles, régler la taille de contexte qui tient dans ta carte graphique et les démarrer avec les agents qui les utilisent.
- **Serveurs MCP et skills** — trouvés sur ton ordinateur ou sur le web, avec des serveurs recommandés et une recherche dans le registre MCP officiel.
- **Problèmes ouverts** — un ! dans l'en-tête quand il y a quelque chose à corriger, chaque problème avec le chemin vers l'endroit où il se corrige, et ce à quoi sert ta configuration.

1.2 Notions clés

Agent	Un programme avec lequel tu parles et qui écrit du code ou fait des tâches, comme Claude Code.
Fournisseur de LLM	L'endroit où tourne le modèle d'un agent : un service cloud (Anthropic, OpenAI, …) ou un moteur de LLM local sur cet ordinateur (llama.cpp, Ollama, …).
Modèle	Le modèle de langage lui-même. Les modèles locaux sont des fichiers, le plus souvent au format GGUF.
Serveur MCP	Un petit programme ou service web qui donne des outils supplémentaires aux agents — lire des fichiers, chercher sur le web, piloter un navigateur. MCP est le Model Context Protocol.
Skill	Un dossier d'instructions (un `SKILL.md`) et parfois de scripts, qui apprend à un agent une tâche précise.

1.3 Rien ne change sans aperçu

Chaque modification des réglages d'un agent est d'abord montrée, ligne par ligne, avant que quoi que ce soit ne soit écrit. L'ancien fichier est gardé en sauvegarde, et la barre qui apparaît après la modification propose **Annuler**. L'app elle-même ne tourne jamais avec des droits d'administrateur ; seule l'installation d'un agent fourni comme paquet du système demande ton mot de passe, dans un terminal que tu vois.

CHAPITRE 2

Installation

2.1 Asterism ASTERISM

Choisis l'une de ces méthodes :

- **Arch Linux (AUR) :** installe le paquet `asterism` avec ton assistant AUR.
- **AppImage :** télécharge `Asterism-<version>-x86_64.AppImage` depuis la page des versions sur `github.com/uBruckhaus/Asterism`, rends-le exécutable et lance-le. L'AppImage utilise le Python 3 de ta distribution avec PyGObject, GTK 4 et libadwaita :

Fedora	`sudo dnf install python3-gobject gtk4 libadwaita`
Ubuntu / Debian	`sudo apt install python3-gi gir1.2-gtk-4.0 gir1.2-adw-1`
Arch	`sudo pacman -S python-gobject gtk4 libadwaita`

- **Depuis le code source :** lance `bin/asterism` dans un clone du dépôt.

L'app apporte ses propres icônes, aucune police d'icônes n'est donc nécessaire. Pour faire tourner des agents dans la fenêtre, installe aussi VTE pour GTK 4 (`vte4`).

2.2 AI Agents Map sur Omarchy OMARCHY

Installe le plugin depuis l'Omarchy Plugin Marketplace ou dans un terminal :

```
omarchy plugin add https://github.com/uBruckhaus/omarchy-ai-agents-map.git --enable
```

Un bouton robot apparaît dans la barre. Tout le reste dont il a besoin est fourni par une installation normale d'Omarchy. Pour faire tourner des agents dans la fenêtre, installe VTE : `sudo pacman -S vte4` .

2.3 Premier démarrage

À la première ouverture, l'app demande si elle doit s'ajouter au lanceur d'applications. Rien n'y est écrit sans ton accord ; tu peux changer cela plus tard dans Réglages › **Dans le lanceur**.

La fenêtre est une fenêtre d'application normale : elle peut être en mosaïque, flottante, déplacée vers un autre espace de travail et atteinte avec `Alt`+`Tab`. ASTERISM Un Asterism neuf s'ouvre sur sa **page d'accueil** : les agents et LLM locaux installés sur cet ordinateur sont des cartes déjà cochées, et **Activer** les met sur la carte (voir Configurer Asterism). OMARCHY Le bouton de la barre ouvre la fenêtre, la ramène au premier plan ou la masque si elle y est déjà. Fermer masque la fenêtre ; le programme continue en arrière-plan pour se rouvrir aussitôt.

> **REMARQUE** Quand un agent tourne dans la fenêtre, la fermeture demande d'abord : **Arrêter et fermer** (ce qui arrête aussi le LLM local de l'agent et libère sa mémoire graphique) ou **Laisser tourner**.

2.4 Retirer l'app

OMARCHY `omarchy plugin remove ubruckhaus.ai-agents-map`. ASTERISM Retire le paquet ou supprime l'AppImage.

Avant cela, désactive Réglages › **Démarrer les LLM locaux avec leurs agents** pour que le veilleur en arrière-plan soit aussi retiré, et Réglages › **Dans le lanceur** pour supprimer l'entrée du lanceur. Ce que tu as modifié avec l'app reste : les réglages des agents, les serveurs MCP, les skills et les LLM locaux installés. Désinstalle d'abord un LLM local (clic droit › **Désinstaller…**) si tu veux qu'il disparaisse. Pour supprimer aussi l'état propre de l'app :

```
rm -rf ~/.config/omarchy/ai-agents-map ~/.config/asterism \
       ~/.local/state/ai-agents-map ~/.cache/ai-agents-map
```

CHAPITRE 3

La fenêtre en un coup d'œil

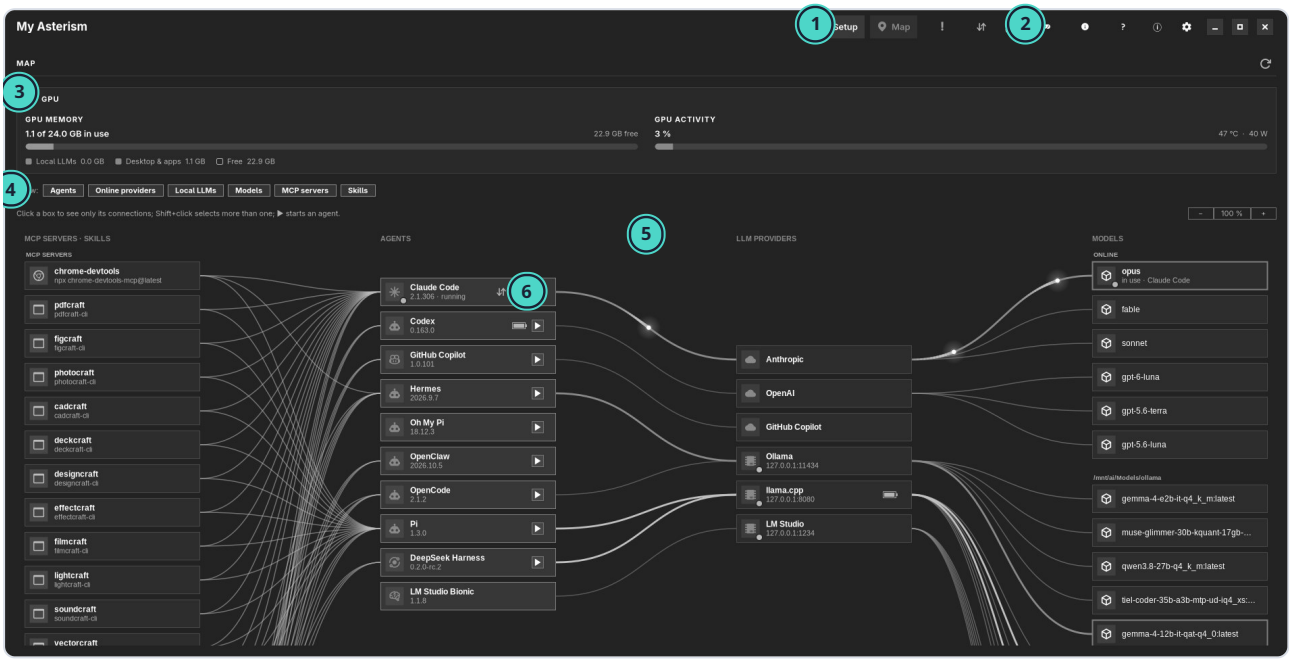


Figure 2 · La fenêtre principale. Les numéros correspondent à la liste ci-dessous.

1. **Configuration** et **Carte** — les deux vues, sur chaque page ; celle affichée est mise en évidence. Configuration présente chaque agent, LLM local, serveur MCP, dossier de skills et modèle sous forme de carte : installer, mettre à jour, se connecter, retirer (voir plus bas).
2. Les symboles de l'en-tête : ↻ actualiser (`F5` : regarder à nouveau, comme au démarrage), **!** problèmes ouverts et mises à jour (rouge : quelque chose à corriger ; la couleur d'accent : seulement des mises à jour et des suggestions ; absent quand il n'y a rien), ⓘ la légende (ce que veut dire chaque point, ligne et marque de la carte), ⇅ le trafic réseau des agents en cours (voir Trafic réseau), les deux étincelles de **Smart Check** (seulement quand Claude Code ou Codex est installé, voir Smart Check), les modules `ASTERISM`, le **i** plein pour Informations (voir Aide), **?** l'Aide (`F1`) et l'engrenage des Réglages (`Ctrl`+`,`). Garde le bouton gauche de la souris appuyé sur un symbole et fais-le glisser sur le côté pour le placer ailleurs : une barre montre où il arrive, et l'ordre est conservé.
3. **GPU** — mémoire GPU et activité GPU : comment la carte graphique est utilisée en ce moment, dans une carte qui se replie (repliée, sa ligne garde les chiffres).
4. Interrupteurs **Afficher** — réduisent la carte aux parties sur lesquelles tu travailles.
5. **La carte** — serveurs MCP et skills ← agents → fournisseurs de LLM → modèles, sur toute la largeur, sous son titre **CARTE**.
6. ▶ dans la boîte d'un agent le démarre dans un terminal dans la fenêtre ; pendant qu'il y tourne, le bouton ramène son terminal.

3.1 Configuration

La carte ne montre que ce qui est configuré : les agents et LLM locaux installés, les serveurs MCP et dossiers de skills que les agents utilisent, et les modèles. Tout le reste se fait dans **Configuration** (le bouton de l'en-tête à côté de **Carte**), une carte par élément, dans des sections qui se replient d'un clic sur leur barre :

- **Comment les éléments arrivent sur ta carte** — une carte repliée tout en haut ; ouvre-la pour cinq étapes numérotées : activer (ou trouver) ce que tu utilises, installer ce qui manque, te connecter à un agent en ligne, relier en dessinant, et ce qui apparaît tout seul.
- **Ce que tu peux en faire** — ce à quoi sert la configuration (aide au quotidien, programmation, images, voix, recherche et écriture), une tuile chacun avec ce qui est là et l'étape suivante ; repliée, sa ligne dit combien sont prêts.
- **Agents IA, LLM locaux** — tous ceux que l'app connaît. Une carte à bordure bleue est installée sur ce PC ; une grise ne l'est pas, et elle a **Installer** là où les installées ont **Désinstaller**. OMARCHY Un clic sur une carte installée ouvre ses détails. ASTERISM Un clic coche ou décoche la carte (✓), puis **Appliquer** active ou désactive son module. Désactiver la retire de la carte ; rien n'est supprimé.
- **Fournisseurs en ligne** ASTERISM — le module des fournisseurs en ligne ; son menu propose **Se connecter à …** pour chaque agent installé qui les utilise.
- **Serveurs MCP, Skills, Modèles** — une carte pour chaque élément installé. Un clic ouvre ses détails (pour un modèle, son menu). La carte **+** à la fin d'une section en ajoute un : **Ajouter un serveur MCP**, **Ajouter des skills**, **Ajouter un modèle**, **Obtenir des extensions**.
- ⋯ ou un clic droit sur une carte ouvre le même menu que sa boîte sur la carte : mettre à jour, se connecter, démarrer ou arrêter, détails, désinstaller ou retirer, déplacer un modèle.
- Un ↑ **version** jaune sur une carte signifie qu'une mise à jour est prête ; son menu propose **Mettre à jour**.
- Un **!** rouge sur une carte signale un problème ouvert à cet endroit ; survole-le pour savoir lequel.
- Une petite batterie sur la carte d'un agent en ligne montre ce qui reste de ses limites (une session, une semaine) : la plus serrée compte, jaune à un quart ou moins, rouge sous un dixième ; survole-la pour chaque limite et sa remise à zéro. Sur la carte d'un LLM local en marche, la batterie est la mémoire GPU encore libre, avec les mêmes couleurs ; survole-la pour voir combien ce LLM local en utilise.
- Le symbole d'actualisation (F5) de la barre d'en-tête regarde à nouveau ce qui est installé, configuré et connecté, et refait les vérifications du démarrage (nouvelles apps, add-ons, mises à jour, problèmes ouverts).

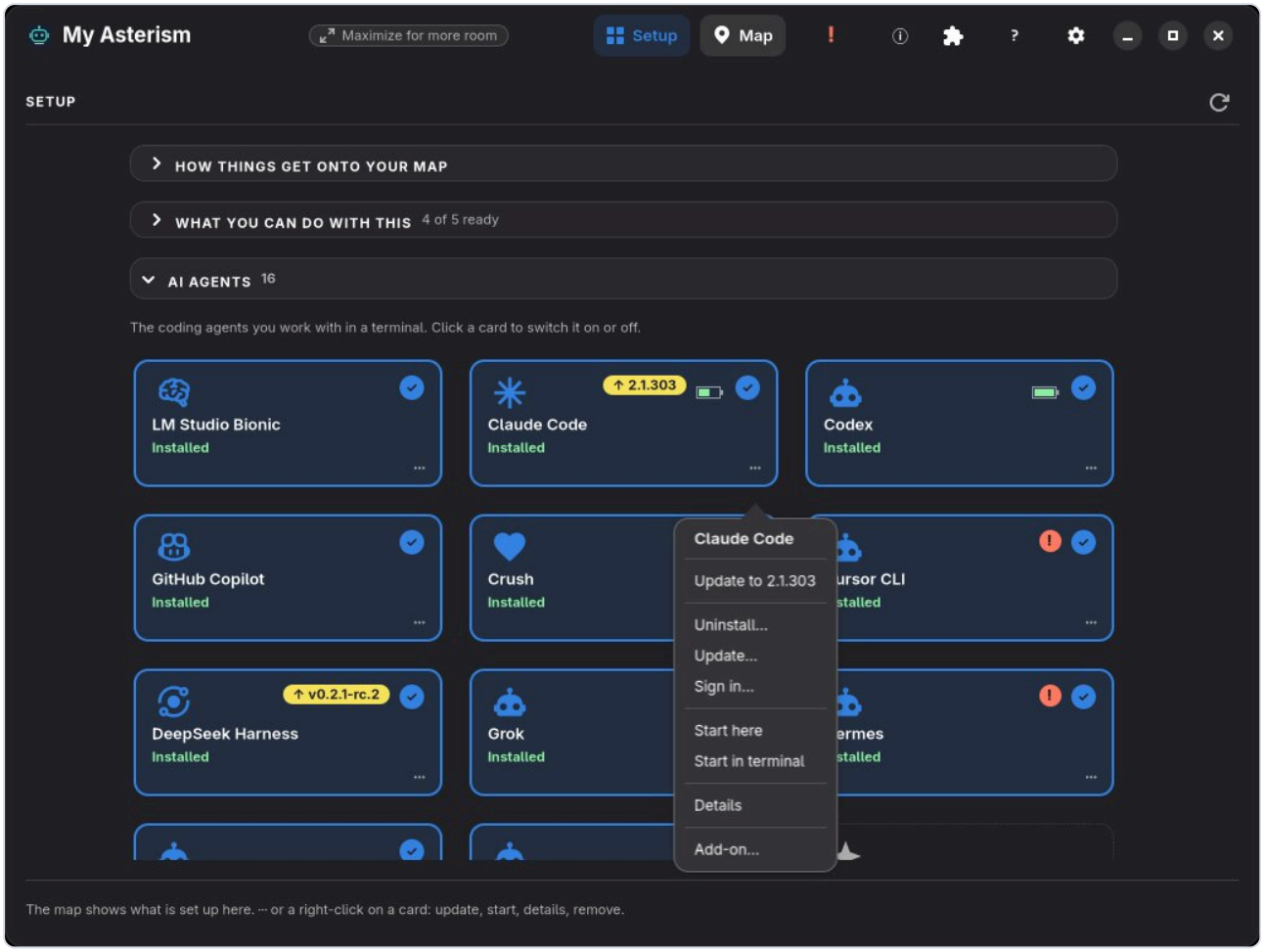


Figure 3 · Configuration : les agents installés ont un cadre bleu, ceux activés un ✓, les mises à jour un badge jaune et les problèmes un rouge ; le menu de Claude Code.

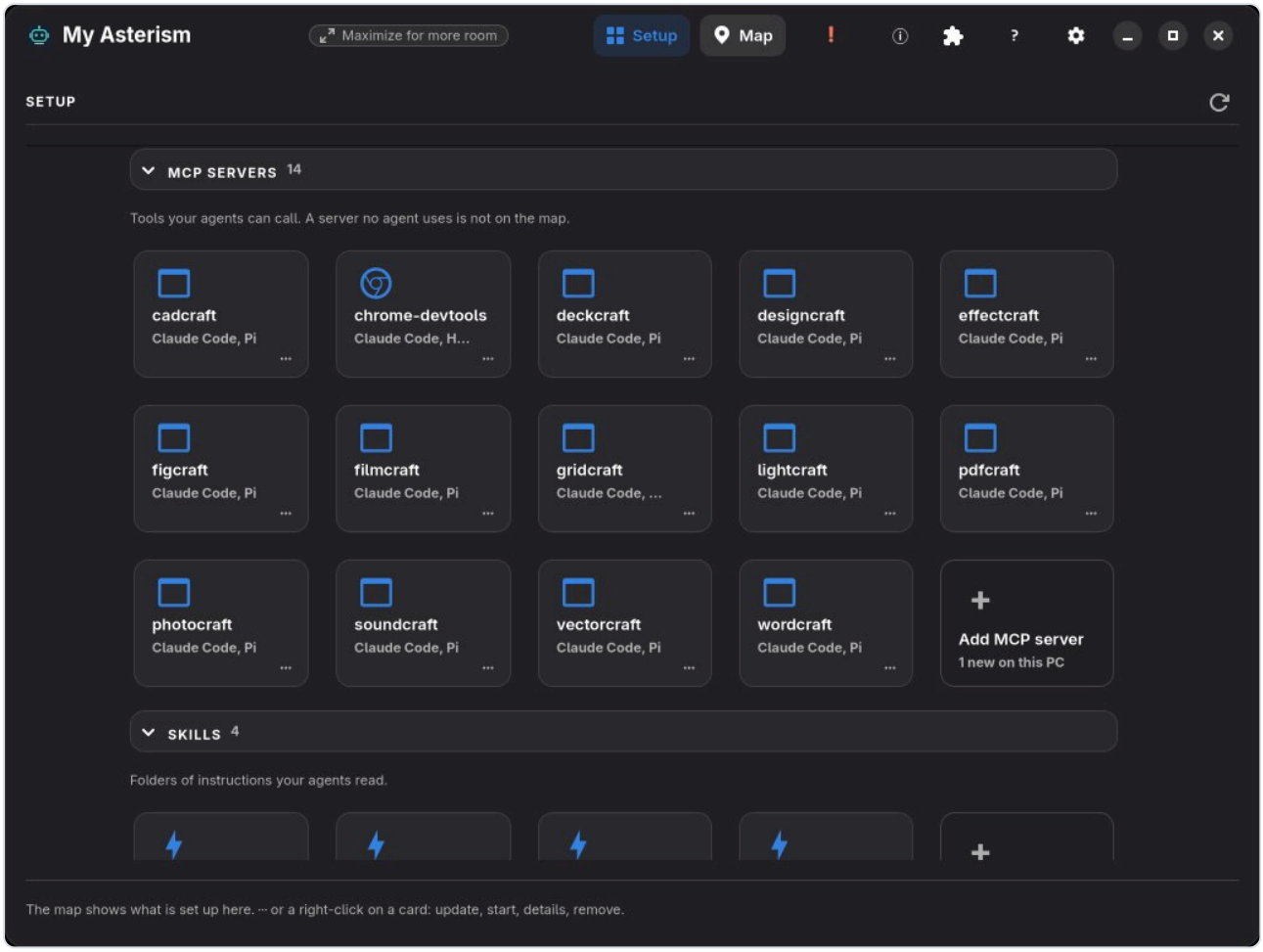


Figure 4 · Serveurs MCP et skills, chaque section avec une carte pour en ajouter.

Il n'y a rien à masquer : ce que tu ne veux pas sur la carte se désactive ou se retire ici. Quand une installation ou une connexion tourne dans un terminal dans la fenêtre, **Retour** (ou **×** sur son onglet) ramène à la page d'où elle est partie.

3.2 Choisir ce que montre la carte

La carte montre tout ce qui est configuré. Les interrupteurs **Afficher** sous la carte GPU laissent de côté des zones entières — par exemple toutes les skills, ou les LLM locaux et leurs modèles — et sont mémorisés.

Dans une fenêtre étroite (la moitié de l'écran ou moins), la carte les choisit elle-même, et une ligne à côté des interrupteurs le dit : tant qu'un agent tourne, agents et fournisseurs (qui travaille par quoi) ; tant qu'aucun ne tourne, fournisseurs et modèles. Un interrupteur cliqué là reste jusqu'à ce que cela change et n'est pas mémorisé ; la fenêtre en taille réelle garde ton propre choix.

3.3 Travailler avec la carte

- **Survole** une boîte pour la mettre en évidence avec tout ce qui y est relié : leurs bordures prennent la couleur d'accent et le reste de la carte s'estompe. Sinon, chaque boîte a la même fine bordure grise.

- **Clique** sur une boîte pour la sélectionner (✓) ; `Maj`+clic en ajoute d'autres. Les boîtes sélectionnées et tout ce qui y est relié gardent la bordure couleur d'accent tant que le pointeur n'est pas sur une autre boîte. `Échap` ou **Effacer la sélection** l'efface.
- L'**épingle** en haut à droite d'une boîte garde ses connexions en évidence pendant que tu regardes ailleurs.
- **Clic droit** sur une boîte pour tout ce que propose sa carte dans Configuration : démarrer ou arrêter, détails, installer, se connecter, modèles, mettre à jour, retirer et plus.
- ▶ dans la boîte d'un agent le démarre dans un terminal dans la fenêtre.
- Les agents restent toujours au centre de la partie visible de la carte pendant le défilement.
- Le chemin actif de chaque agent — agent → le fournisseur qu'il utilise → son modèle — est dessiné en gras. Pendant qu'un agent travaille, une lumière parcourt cette ligne.
- L'**encadré des statistiques d'utilisation** : pendant qu'un agent travaille, de petits nombres sont écrits le long de sa ligne vers le fournisseur, dans la couleur de la ligne : la vitesse de sa dernière réponse (**t/s**, jetons par seconde), la taille de son contexte (**1M**) et la part utilisée (**%**, jaune à partir de 75 %, rouge au-delà de 90 %, comme les batteries). Ils viennent des propres fichiers de session de l'agent (nombres uniquement ; Claude Code, Codex et Pi). Ce qu'il affiche et sa taille de texte se règlent dans les Paramètres.
- Une ligne rouge en pointillés relie un agent à un fournisseur pour lequel il est configuré mais qu'il n'utilise pas en ce moment (aucun de ses modèles en usage).
- Quand des agents utilisent des modèles différents d'un même LLM local, les lignes vers ces modèles sont violettes : il les garde chargés en même temps, ou les alterne (voir Modèles en même temps).
- La boîte d'un agent en ligne a la même batterie que sa carte dans Configuration : ce qui reste de ses limites ; la boîte d'un LLM local en marche, la mémoire GPU encore libre.
- Un agent payé au token — connecté avec une clé d'API, donc sans limites de forfait pour une batterie — affiche à la place ce que coûtent aujourd'hui ses sessions actives aux prix de l'API, sous la forme d'un petit **« 3,40 $ aujourd'hui »** sur sa case et sa carte : vert, jaune à partir de 75 % de la limite du jour et rouge au-delà (Réglages › **Coûts**), mis à jour chaque minute. Ses détails montrent tout le coût d'API.
- Quand un programme absent de la carte utilise un LLM local, la ligne de ce LLM vers son modèle devient épaisse et prend sa propre couleur (violet ou cyan, celle qui ressort le mieux des autres couleurs du thème), et une lumière la parcourt. Voir Utilisé par d'autres apps.
- Quand un agent en a lancé un autre — Claude Code exécutant `pi -p …` ou `codex exec …` depuis son shell, par exemple —, une **flèche** longe la gauche de la carte des agents, de l'agent qui utilise l'autre vers celui qu'il utilise, et la lumière la parcourt ; son infobulle nomme les deux. Un agent peut en utiliser plusieurs à la fois, chacun avec sa propre flèche. Tant qu'il est utilisé, l'agent utilisé a aussi ses lignes vers son fournisseur et jusqu'à son modèle dans la couleur de la flèche, et la lumière y court dans cette couleur : tu vois d'un coup d'œil quel fournisseur et quel modèle travaillent pour qui.
- Le symbole d'actualisation (`F5`) de la barre d'en-tête regarde à nouveau ce qui est installé, configuré et connecté, et refait les vérifications du démarrage (nouvelles apps, add-ons, mises à jour, problèmes ouverts).
- Tant que la première ligne n'est pas tracée, une carte au-dessus de la carte montre comment en trois étapes ; **Voir comment** lance l'animation de l'Aide. Après la première connexion, ou avec ×, elle

disparaît pour de bon.

- Zoom avec **− 100 % +**, `Ctrl`+molette ou `Ctrl`+`+` / `-` / `0`.
- Les boîtes sont dans une carte par catégorie (agents, fournisseurs en ligne, LLM locaux, une carte par dossier de modèles, serveurs MCP, skills). Fais glisser une carte par son en-tête pour la poser n'importe où sur la carte, et une boîte dans sa carte pour changer l'ordre.
- **Clique sur l'en-tête d'une carte** pour la replier jusqu'à son en-tête (▸ à droite de l'en-tête) et encore pour l'ouvrir (▾). Les lignes des boîtes d'une carte repliée s'arrêtent à son en-tête : tu vois toujours ce qui y est relié (les flèches entre agents sont masquées tant que la carte des agents est repliée). Ouvrir une carte écarte les cartes que tu as posées autour.
- L'**épingle** à droite du zoom garde la disposition de la carte — où tu as posé les cartes, l'ordre des boîtes et les cartes repliées — après **Actualiser** (`F5`) et un redémarrage. Non épinglée, comme au départ, la carte revient à sa disposition par défaut à chaque actualisation et démarrage ; tes changements restent jusque-là. **Disposition par défaut** remet tout en place d'un coup.

> **ASTUCE** Dans une fenêtre étroite (moins d'environ 1300 pixels, à peu près un demi-écran), la carte défile latéralement.

3.4 Problèmes ouverts

Quand il y a quelque chose à corriger, un **!** rouge apparaît parmi les symboles de l'en-tête ; son info-bulle dit combien, par exemple *2 problèmes ouverts · 4 suggestions*. Un clic ouvre **Problèmes ouverts** : une ligne chacun — un agent non connecté, un LLM local sans modèles, une liaison qui ne fonctionne pas, peu de place libre pour les modèles — avec un bouton vers l'endroit où il se corrige (les détails de l'agent ou du LLM local, la carte avec cette ligne mise en évidence, Configuration, Ranger). Les mêmes problèmes apparaissent comme un **!** rouge sur leurs cartes dans Configuration. Les conseils et ce qui fonctionne se trouvent sur les cartes et pages de détails elles-mêmes, et ce à quoi sert la configuration est dans Configuration (**Ce que tu peux en faire**). La vérification ne fait que lire ; elle ne modifie, ne démarre et n'envoie jamais rien, et ne montre rien tant que la carte est vide.

3.5 Mises à jour et suggestions

Sous les problèmes, sur la même page, **Mises à jour et suggestions** liste ce qu'il y a à dire : de nouveaux programmes ou skills sur ce PC, ce qui est sur ce PC mais pas sur la carte, et les mises à jour des agents et LLM locaux installés, chacun avec son bouton (**Ajouter un serveur MCP…**, **Configuration**, **Mettre à jour**). Quand il n'y a que celles-ci et aucun problème, le **!** de l'en-tête prend la couleur d'accent. Les mises à jour sont cherchées en arrière-plan à l'ouverture de la fenêtre, au plus toutes les six heures. La recherche elle-même n'installe rien ; une mise à jour ne commence que quand tu cliques sur **Mettre à jour**.

Avec Réglages › **Vérifications IA quotidiennes** activé, les mises à jour de serveurs MCP qu'un agent a trouvées sur le web sont listées ici aussi, avec **Page de la version** et **Le faire avec …** ; une fois la nouvelle version sur ce PC, la ligne propose **L'utiliser…** à la place (voir Mises à jour trouvées).

CHAPITRE 4

Agents

4.1 Les détails d'un agent

Chaque agent de programmation a une carte dans Configuration, les installés avec leur version. Clique sur la carte d'un agent installé (ASTERISM ou ··· › **Détails**), ou clic droit sur sa boîte de la carte › **Détails**. En haut, une ligne dit si l'agent est prêt (et sinon pourquoi) et s'il tourne en ce moment ou quand il a servi pour la dernière fois. En dessous, chaque partie est une carte repliable : un clic sur son titre l'ouvre ou la ferme. Elles commencent repliées, chacune avec un résumé d'une ligne et des badges ; une partie que tu ouvres reste ouverte sur les pages de détails suivantes.

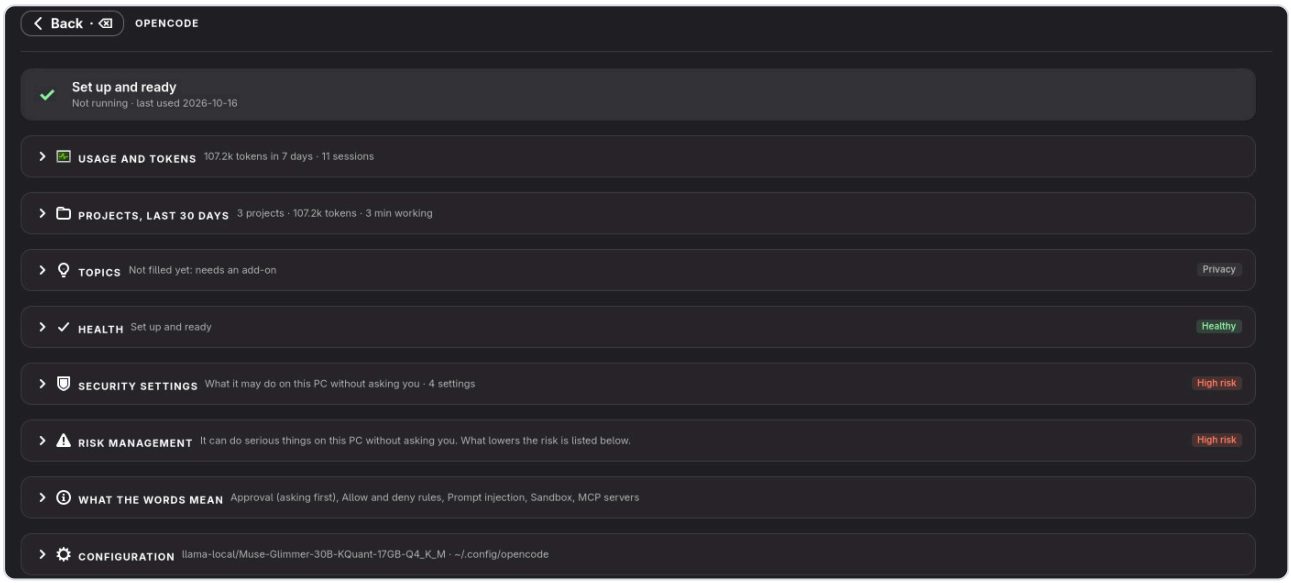


Figure 5 · Les détails d'un agent : chaque partie repliée, avec son résumé et ses badges.

Carte	Ce qu'elle montre
Utilisation et jetons	Ce que l'agent enregistre : jetons et sessions d'aujourd'hui et des sept derniers jours, l'abonnement, un graphique des jetons par jour, les limites avec leur heure de remise à zéro et les jetons par modèle.
Coût d'API / **Valeur API**	Ce que coûtent ses tokens aux prix d'API des fournisseurs : aujourd'hui, les 7 et 30 derniers jours et, par modèle, la répartition en entrée, sortie, lecture du cache et écriture du cache (tokens et montant). **Coût d'API** pour un agent payé au token (avec sa part de la limite du jour), **Valeur API** pour un agent avec forfait (le forfait le couvre). Voir Coût d'API.
Projets, 30 derniers jours	Par dossier de projet : temps de travail, sessions, jetons, exécutions de sous-agents, lignes modifiées et la valeur API.
Sujets (Confidentialité)	De quoi parlaient les sessions. Cela va au-delà des chiffres, c'est pourquoi la carte porte la marque **Confidentialité** et n'est remplie que par un module (voir ci-dessous) ; le module nomme la carte et son badge dit ce qu'il lit.

Carte	Ce qu'elle montre
État	Chaque vérification de la configuration avec un ✓, ou ce qu'il faut faire ensuite : se connecter, installer ou démarrer son LLM local, télécharger un modèle, ajouter des serveurs MCP ou des skills.
Réglages de sécurité, **Gestion des risques**, **Ce que veulent dire les mots**	Ce que l'agent peut faire sans te demander et ce qui réduit le risque (voir Bilan de santé et de sécurité).
Configuration	Les réglages de l'agent, ses serveurs MCP, ses skills et son dossier de réglages.

Les projets sont affichés pour Claude Code, Codex, Pi, Oh My Pi, OpenCode, Hermes, DeepSeek Harness, Goose, Gemini CLI et OpenClaw : temps de travail, sessions, jetons, exécutions de sous-agents, lignes modifiées et ce que coûteraient les jetons au tarif API. Les chiffres viennent des propres fichiers de session ou de la base de données de chaque agent ; aucun texte n'est lu.

Coût d'API. Les montants sont une estimation : les nombres de tokens que garde l'agent (entrée, sortie, lecture et écriture du cache, par modèle) aux prix d'API publiés par les fournisseurs — Anthropic, OpenAI, DeepSeek, xAI et Google, avec la date à laquelle ils ont été relevés. Une session compte le jour de sa dernière réponse ; les modèles locaux ne coûtent rien. Crush ne garde pas de nombres de tokens qui s'additionnent : sa carte montre donc le coût que Crush indique lui-même. Pour un agent qui ne garde aucun nombre de tokens sur le PC (Cursor CLI, Antigravity, Muse), la carte dit que le coût est inconnu ; la page de compte de son fournisseur l'indique. Les montants s'affichent en dollars américains, ou en euros au cours de référence quotidien de la Banque centrale européenne (Réglages › **Coûts**).

ASTERISM Avec le module **Insights** (voir Modules) la carte **Sujets** montre d'abord quels sujets ont pris le plus de jetons ces 30 derniers jours (**Le plus de jetons** : les sessions de même titre forment un seul sujet, chacun avec sa part de tous les jetons), puis les titres que l'agent a donnés lui-même à ses sessions, par projet (**Par projet**), et propose **Résumer avec Claude…** (si Claude Code est installé). Ses jetons viennent des fichiers de session de l'agent, sans les lectures en cache, et peuvent donc être bien plus bas que dans **Utilisation et jetons**, qui peut compter les lectures en cache. Avant tout envoi, tu vois exactement ce qui sera envoyé : les titres et les chiffres, jamais les conversations.

4.2 Démarrer et arrêter des agents

- ▶ dans la boîte de l'agent ou **Lancer ici** dans son menu démarre l'agent dans un terminal dans la fenêtre, à la place de la carte. Cela nécessite VTE (`vte4`).
- **Lancer dans un terminal** l'ouvre dans sa propre fenêtre de terminal.
- Pendant qu'il tourne dans la fenêtre : **Afficher le terminal** et **Arrêter**. Arrêter met fin à l'agent et à tout ce qu'il a démarré, comme ses serveurs MCP.
- **Terminaux…** liste où d'autre l'agent tourne : *Ici, dans cette fenêtre*, chaque *Fenêtre de terminal* (un clic la met au premier plan) et chaque instance *En arrière-plan* — une sans fenêtre, comme `claude -p …` lancé par un script ou par un autre agent. Un clic sur une instance en arrière-plan ouvre un onglet de terminal qui montre ce qu'elle écrit, au fur et à mesure : questions, réponses et outils qu'elle utilise (depuis son fichier de session, ou sa sortie quand celle-ci va dans un fichier). L'onglet se termine quand l'instance se termine. C'est le seul endroit où l'app montre du texte de conversation, et seulement pour l'instance sur laquelle tu cliques.

- **Arrêter…** — aussi le petit bouton d'arrêt à gauche de ▶ dans la boîte de l'agent — arrête une de ses instances en dehors de cette fenêtre. Choisis-la dans la liste, qui indique où et quand chacune a été lancée. Elle est priée de se terminer (SIGTERM) et, si elle est encore là au bout de 3 secondes, terminée. Seul ce processus précis est arrêté : s'il s'est terminé entre-temps et que son numéro de processus appartient maintenant à un autre programme, rien ne se passe.

Quand des agents ont des terminaux dans la fenêtre, une rangée d'onglets au-dessus de la carte montre un onglet par terminal, avec un point pour *travaille*, *tourne* ou *terminé*. Un clic sur un onglet affiche ce terminal ; × ferme un terminal terminé.

Un agent ouvert n'importe où sur l'ordinateur reçoit un point vert et *tourne*, et la boîte du modèle avec lequel il tourne indique *utilisé · <agent>*.

4.3 Trafic réseau

La boîte et la carte d'un agent en cours montrent ce qu'il reçoit et envoie (↓ entrant, ↑ sortant). Clique sur cette ligne, ou sur ⇅ dans l'en-tête pour tous les agents en cours, pour ouvrir sa page réseau : **Maintenant**, **Les 5 dernières minutes** en graphique (entrant en vert, sortant en bleu), **Où ça va** — API en ligne, LLM locaux, ce PC et ton réseau local, chacun avec sa part — et **Programmes** : l'agent lui-même et les programmes qu'il a lancés, comme ses serveurs MCP. C'est compté à partir des connexions TCP de l'agent pendant que l'app est ouverte (sans droits d'administrateur ; l'UDP n'est pas compté) et ça reste sur ce PC.

4.4 Sous-agents

Claude Code et OpenCode peuvent confier des parties d'une tâche à des **sous-agents**. Tant qu'un des sous-agents d'un agent travaille, sa case sur la carte montre une petite marque en arbre à côté de ⇅ ; s'il n'en a que de terminés dans la liste, la marque est atténuée. Cliquez dessus pour ouvrir la page **Sous-agents** de l'agent : une carte à elle, disposée comme la grande, avec la carte de l'agent à gauche et une carte **Sous-agents** à droite. Chaque case montre la tâche du sous-agent, son type, le dossier où travaille sa session, et **au travail** ou **terminé il y a … minutes**. La lumière parcourt les lignes vers ceux qui travaillent ; ceux qui ont fini sont atténués. Survolez une case pour lire toute sa tâche.

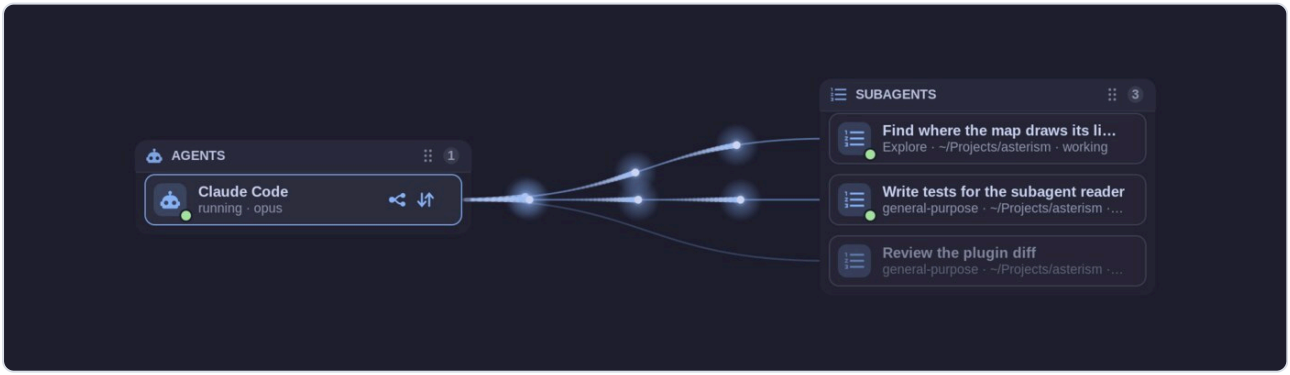


Figure 6 · Les sous-agents d'un agent : deux au travail, un terminé.

Combien de temps les sous-agents terminés restent dans la liste (30 minutes) et combien au plus (10) se règle dans Réglages › **Sous-agents** ; ceux qui travaillent sont toujours affichés. La page ne lit que le type de chaque sous-agent et la description d'une ligne de sa tâche, dans les fichiers de l'agent lui-

même (les dossiers de session de Claude Code, la base de données d'OpenCode), jamais ce que disent les sous-agents.

4.5 Configurer…

Clic droit sur un agent ou un LLM local › **Configurer…** ouvre sa propre configuration dans un onglet de terminal de la fenêtre, pour ce que l'app ne règle pas elle-même — par exemple `hermes setup` ou le menu d'Ollama ; LM Studio ouvre sa fenêtre. Un programme sans commande de configuration affiche son `--help`. Quand c'est fini, la carte regarde de nouveau et montre ce qui a changé, et on propose de redémarrer un LLM local en cours.

4.6 Se connecter

Se connecter… dans le menu d'un agent (dans Configuration ou sur la carte) montre ce dont l'agent a besoin et ses propres façons de se connecter :

- **Un abonnement ou un compte** — la connexion propre à l'agent tourne dans un terminal dans la fenêtre (par exemple `claude auth login` ou `codex login`). Ton mot de passe ou la connexion dans le navigateur restent entre toi et l'agent ; l'app ne les voit jamais. Quand l'agent le demande, le terminal dit quoi taper (Pi et Copilot : `/login`).
- **Une clé API** — pour les agents qui en lisent une : la clé va dans un champ mot de passe, puis directement dans la propre commande de l'agent (Codex la stocke lui-même) ou dans le fichier que lit l'agent (Gemini : `~/.gemini/.env`, lisible par toi seul). Elle n'atterrit jamais dans une ligne de commande, un journal ou les réglages de l'app, et l'app ne la garde pas. La facturation API de Claude Code est sa propre connexion à l'Anthropic Console, donc aucune clé ne passe par l'app.

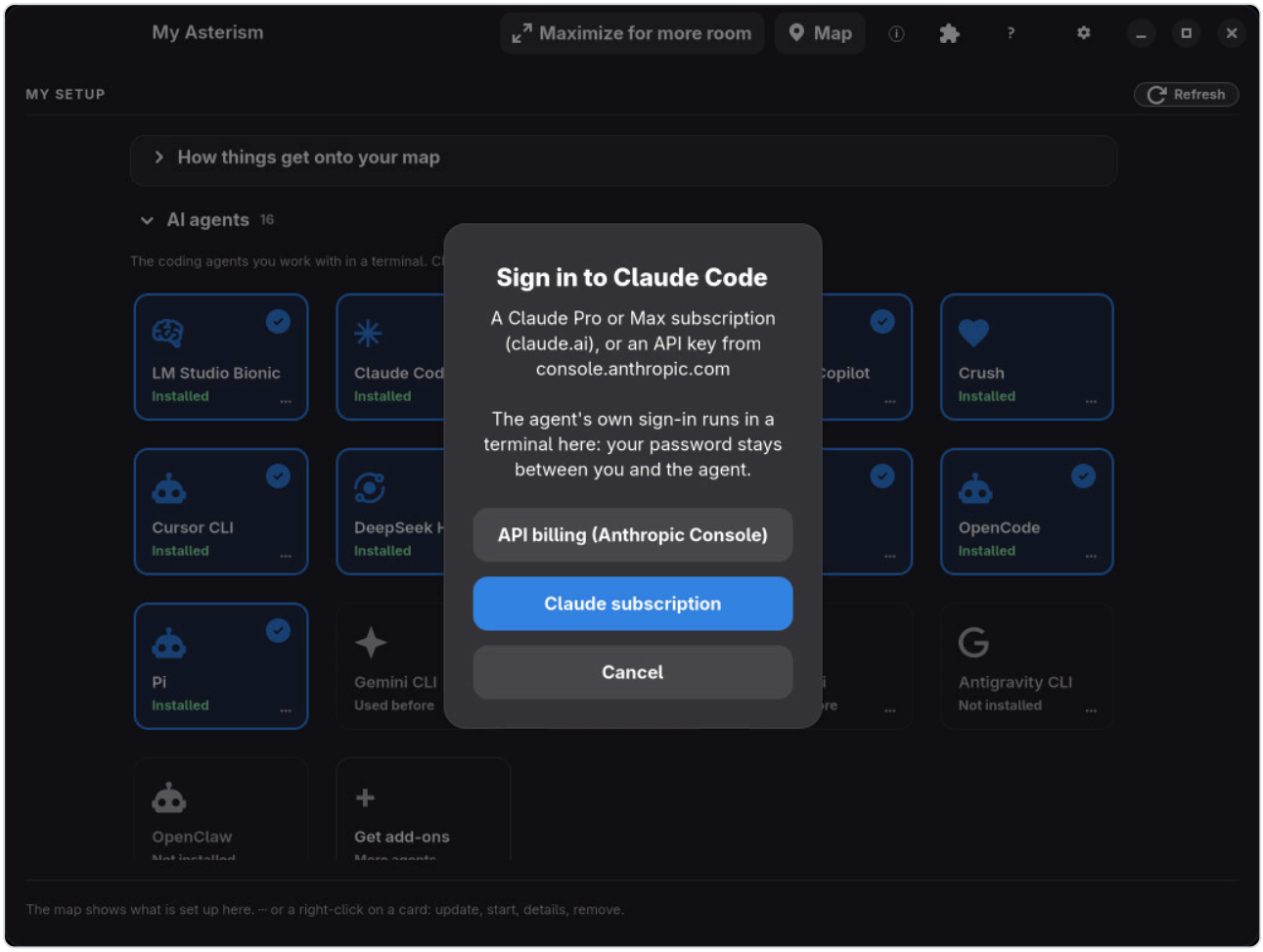


Figure 7 · Se connecter à Claude Code : un abonnement ou la facturation API, tous deux avec la connexion propre à Claude Code.

Ensuite, la carte montre le fournisseur en ligne de l'agent et ses modèles.

4.7 Mettre à jour des agents

Quand une version plus récente d'un agent installé existe, sa carte dans Configuration reçoit un ↑ **version** jaune, et **Mises à jour et suggestions** (le **!** de l'en-tête) la mentionne. **Mettre à jour** lance la mise à jour de la façon dont l'agent a été installé — par exemple `mise upgrade <outil>` ou `claude update` — dans sa propre fenêtre de terminal pour que tu puisses la suivre.

4.8 Agents que l'app peut installer

Quatre agents peuvent être installés, mis à jour et désinstallés depuis la carte (clic droit sur l'agent › **Installer…**) :

- **Goose** (`goose` , de l'Agentic AI Foundation, auparavant Block), sa version GitHub via mise. Il fonctionne avec de nombreux fournisseurs et avec des LLM locaux ; **Configurer…** lance `goose configure` .
- **DeepSeek Harness** (`dsh` , aperçu pour développeurs), depuis npm. Il peut utiliser des LLM locaux.

- **Antigravity CLI** (`agy` , Google), depuis le manifeste de versions de Google ; le téléchargement est vérifié avec sa somme SHA-512.
- **LM Studio Bionic** ne peut pas être installé d'ici (aucune somme de contrôle publiée) ; récupère-le sur lmstudio.ai. Il utilise toujours le moteur et les modèles de LM Studio.

4.9 Nouveaux agents

Quand un agent que l'app ne connaît pas encore apparaît, elle demande une fois à un agent installé (Claude Code, sinon Codex) de chercher sur le web où cet agent range ses skills et ses serveurs MCP. La réponse est vérifiée avant d'être utilisée, et rien de ce qu'elle contient n'est jamais exécuté. Réglages › **Rechercher les nouveaux agents** désactive cela.

CHAPITRE 5

LLM locaux

Un moteur de LLM local fait tourner des modèles de langage sur ta propre carte graphique, si bien que les requêtes ne quittent jamais l'ordinateur. L'app les installe, les démarre et les arrête pour toi.

5.1 Installer un LLM local

Clic droit sur un LLM local gris (non installé) de la carte › **Installer…**. Tu choisis le dossier du programme, le dossier des modèles (celui de tes autres LLM locaux est proposé) et s'il démarre à l'ouverture de session. Il est installé pour ton utilisateur seulement, avec un service utilisateur, donc aucun mot de passe n'est nécessaire. Chaque téléchargement doit correspondre à la somme SHA-256 publiée par sa version avant d'être décompressé ou exécuté.

Moteur	Remarques
Ollama	La version officielle (avec ROCm pour les cartes AMD).
llama.cpp	La compilation de ton choix (Vulkan, ROCm, CUDA, CPU), comme routeur de modèles : n'importe quel GGUF du dossier des modèles, un seul chargé à la fois.
LM Studio	Pas installé d'ici (aucune somme de contrôle publiée) ; récupère-le sur lmstudio.ai. Une fois installé, il est affiché et géré.
vLLM	Pour les cartes NVIDIA, dans son propre environnement Python.
Jan	Son AppImage, décompressé, avec le moteur de Jan comme service d'arrière-plan pour les agents.
KoboldCpp	La compilation officielle en un seul fichier, comme routeur de modèles.

Installé ainsi, un LLM local propose aussi **Mettre à jour…** (la version précédente est gardée en secours), **Dossier des modèles…** et **Désinstaller…**, qui retire le programme, la commande, le service et les réglages, mais jamais les modèles. Les LLM locaux que tu as installés toi-même sont affichés, mais jamais modifiés.

5.2 Démarrer et arrêter avec les agents

Connecte une fois un agent à un LLM local, puis démarre simplement l'agent — même app fermée ou après un redémarrage. Son LLM local démarre avec lui, et quand le dernier agent qui l'utilise se termine, le LLM s'arrête environ cinq secondes plus tard, ce qui libère à nouveau sa mémoire graphique.

Un petit service d'arrière-plan s'en charge, `ai-agents-map-providers.service`. Il est mis en place avec la première connexion à un LLM local et retiré quand la dernière disparaît ou quand tu désactives Réglages › **Démarrer les LLM locaux avec leurs agents**.

> **REMARQUE** KoboldCpp met quelques secondes à démarrer, plus longtemps que certains agents n'attendent. Il démarre donc à l'ouverture de session, et quand le dernier agent se termine, seul son modèle est déchargé.

5.3 Les détails d'un LLM local

Clique sur la carte d'un LLM local dans Configuration, ou clic droit sur sa boîte de la carte › **Détails**. Sous les tuiles (modèles, agents, adresse) viennent les mêmes cartes repliables que pour les agents : **Utilisation et jetons** — ce que tes agents ont envoyé par lui ces derniers jours, par agent et par modèle, d'après les propres chiffres de session des agents et associé par nom de modèle —, puis État, Gestion des risques, Ce que veulent dire les mots et Configuration.

5.4 Mémoire GPU et libérer la mémoire GPU

Dans la carte **GPU** au-dessus de la carte, **Mémoire GPU** montre combien est utilisé, réparti entre LLM locaux, bureau et applications, et libre ; **Activité GPU** montre la charge, la température et la puissance. La boîte de chaque LLM local montre combien de mémoire il occupe, et les modèles chargés reçoivent un point vert.

Clic droit sur un LLM local pour **Démarrer**, **Arrêter**, le démarrage à l'ouverture de session et **Libérer la mémoire GPU** : ses modèles sont déchargés, il continue de tourner et en recharge un quand on le lui demande.

Un LLM local qui occupe de la mémoire GPU reçoit un bouton d'arrêt rouge sur sa case. Il demande d'abord : **Libérer la mémoire GPU** décharge ses modèles ; **Arrêter** l'arrête. Pour LM Studio, le bouton est **Arrêter le serveur** : seul son serveur s'arrête, l'application reste ouverte avec ses modèles chargés. Un modèle chargé par un LLM local mais utilisé par aucun agent lui est relié par une ligne claire continue.

LM Studio peut être ouvert sans son serveur. Sa case affiche alors *serveur arrêté* avec un point jaune, les agents ne peuvent pas l'atteindre et son menu propose **Démarrer le serveur** (`lms server start`).

5.5 Modèles en même temps

llama.cpp, LM Studio et Ollama peuvent garder plusieurs modèles chargés. Quand tu connectes des agents à des modèles différents d'un même LLM local, l'app règle combien il en garde chargés en même temps sur le nombre qu'utilisent ses agents — avec le même aperçu, la même sauvegarde et la même Annulation que la connexion — et propose de le redémarrer s'il tourne, car il lit ce nombre au démarrage. Là où ce nombre ne relève pas de l'app (une installation système, ou ta propre commande de démarrage), une note dit où le régler.

Quand les modèles ensemble ne tiennent pas dans la mémoire du GPU, un avertissement le dit : le LLM local les alterne alors, ce qui est lent, ou une partie d'un modèle tourne sur le processeur. Le démarrage d'un agent vérifie la même chose : si charger son modèle demande plus de mémoire GPU qu'il n'en reste, on te le demande d'abord, avec ce qui occupe la mémoire en ce moment — **Libérer la mémoire GPU, puis démarrer**, **Démarrer quand même** ou **Annuler**.

5.6 Utilisé par d'autres apps

Les LLM locaux ne servent pas qu'aux agents. Un plugin comme Video Trans, une app de chat ou ton propre script peuvent aussi leur envoyer des requêtes. L'app le remarque et le montre :



Figure 8 · Un programme absent de la carte (ici `curl`) qui génère avec llama.cpp : la ligne et la boîte cyan (thème Tokyo Night). Le propre chemin de Claude Code vers opus reste dans la couleur d'accent.

- **Sur la carte**, la ligne du LLM local vers le modèle avec lequel il génère devient épaisse et prend sa propre couleur, une lumière la parcourt, et la boîte du modèle reçoit un point de cette couleur et dit qui l'utilise, par exemple *utilisé · Video Trans*.
- **Le menu contextuel du LLM local** et sa page ⓘ **Détails** indiquent *Utilisé par Video Trans · maintenant* ou il y a combien de secondes il a servi pour la dernière fois.
- La légende ⓘ montre les deux marques.

L'app regarde lesquels de tes propres programmes ont une connexion ouverte vers le LLM local. Elle ne lit jamais ce qu'ils envoient. Un plugin apparaît sous son propre nom, un agent sous le sien, et tout le reste sous le nom de son programme. L'app elle-même et LLM Monitor, qui ne lisent que l'état, ne comptent pas.

> **REMARQUE** La lumière ne se voit que pendant que le programme génère et s'éteint environ dix secondes après sa dernière requête. Un programme ouvert mais inactif — Video Trans sans parole à traduire — ne montre rien. Un programme reste dans le menu une demi-minute après sa dernière requête.

CHAPITRE 6

Modèles

Clic droit sur un LLM local installé › **Modèles…**, ou clique sur **Ajouter un modèle** dans **Configuration**. La page a une liste à gauche et le modèle choisi à droite, dans trois onglets.

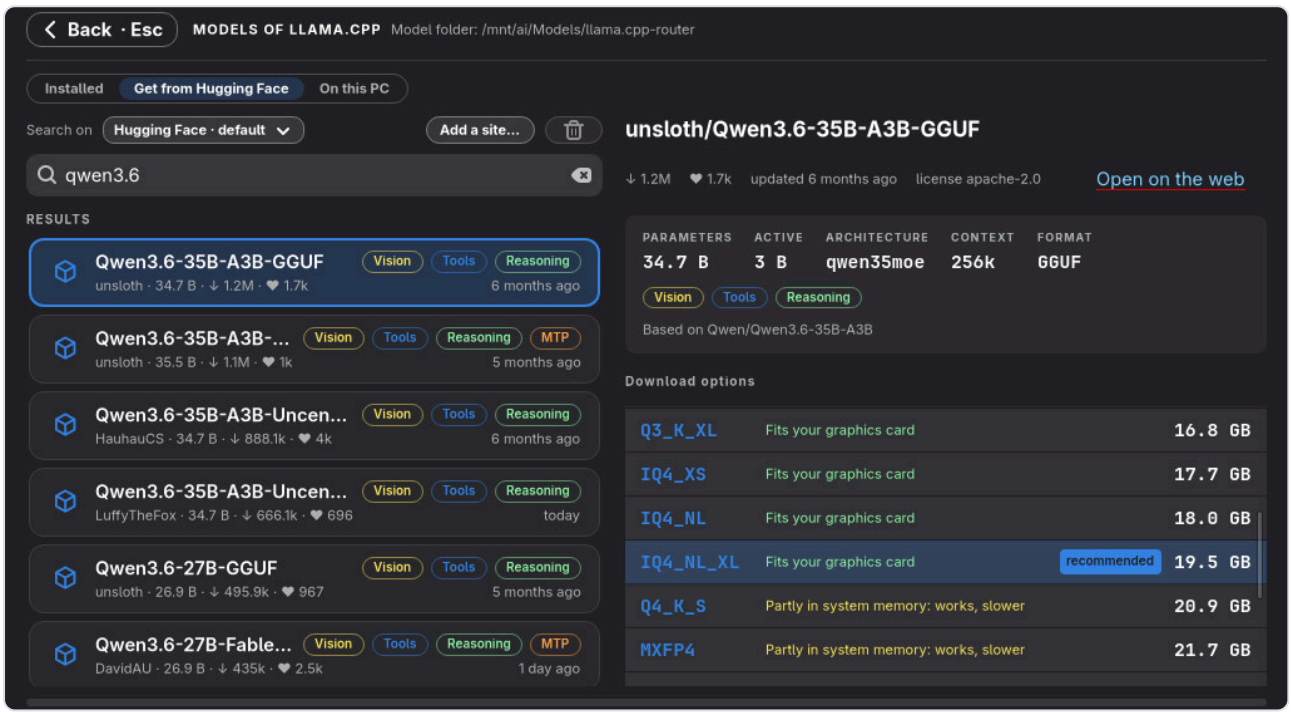


Figure 9 · Récupérer depuis Hugging Face : ce que sait faire un modèle et quelle taille tient dans ta carte graphique.

6.1 Récupérer depuis Hugging Face

Parcours les modèles de chat GGUF populaires ou fais une recherche. Chaque résultat montre l'auteur, la taille, les téléchargements, les j'aime, l'ancienneté et ce qu'il sait faire : **Vision**, **Outils**, **Raisonnement**, **MTP**. La page du modèle montre les paramètres, l'architecture, le contexte, la licence et la fiche du modèle. **Ce que signifie le nom** explique les parties du nom, comme *35B-A3B* ou *Q4_K_M*.

Options de téléchargement liste chaque taille avec la façon dont elle tient : dans la carte graphique, en partie en mémoire vive (fonctionne, mais plus lentement) ou probablement trop grande. Le meilleur choix est déjà fait ; **Télécharger** le récupère dans le dossier des modèles. Pour Ollama, qui télécharge par le nom de la taille, seules les tailles qu'il peut télécharger sont proposées : un fichier découpé, un fichier dans un sous-dossier ou un fichier que Hugging Face refuse pour Ollama est grisé, avec la raison, et jamais recommandé.

Les téléchargements tournent en arrière-plan et apparaissent dans une bande en haut de la page principale. Si la connexion tombe, ils réessaient et reprennent. Si l'app a été fermée pendant un téléchargement, il est listé comme interrompu, avec **Reprendre** ou **Abandonner**.

6.2 Demander à un agent quel modèle convient

Vous ne savez pas quel modèle prendre ? Quand un agent pour les vérifications IA est configuré (Réglages › **Agent pour les vérifications IA**), la carte **Demander à un agent** sous la recherche accepte une question formulée avec vos mots — par exemple *un modèle de code avec un long contexte qui tient dans ma carte graphique*. **Demander** (ou `Ctrl` + `Entrée`) l'envoie, avec votre carte graphique, sa mémoire, la mémoire de l'ordinateur et les modèles que vous avez déjà, à cet agent, qui cherche sur le web ; pendant ce temps, un indicateur animé et le temps écoulé s'affichent. Au bout d'une à deux minutes, la carte affiche son bref résumé, et les modèles proposés apparaissent dessous dans la liste des résultats, sous le titre *Proposé par …* : chacun trouvé sur Hugging Face, avec sa taille et la raison pour laquelle il convient ; un clic ouvre sa page et ses options de téléchargement comme pour tout résultat de recherche. Une suggestion absente de Hugging Face est nommée sous le titre. Il propose jusqu'à 10 modèles ; **Suggestions**, à droite de la ligne de l'agent, règle un autre nombre (5, 10, 15 ou 20). Cela utilise les jetons de l'agent ; la question n'est pas conservée, et la page de chaque LLM local pose sa propre question.

6.3 Installés

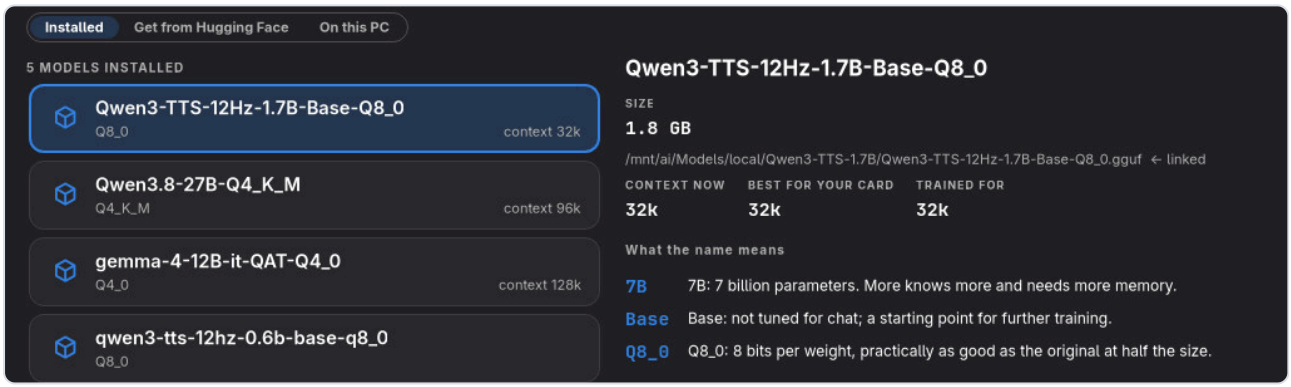


Figure 10 · Installés : contexte actuel, le meilleur pour ta carte et celui de l'entraînement ; ce que signifie le nom ; Retirer.

Chaque modèle montre sa taille, son fichier et son contexte — *actuel*, *le meilleur pour ta carte* et *entraîné pour*. **Retirer** demande d'abord et déplace les fichiers dans la corbeille, d'où tu peux les restaurer. Un modèle qui n'est qu'un lien ne perd que le lien.

6.4 Taille de contexte

Le contexte, c'est la quantité de texte que le modèle peut garder en tête à la fois. L'app calcule, à partir du fichier du modèle lui-même, le plus grand contexte qui tient dans ta carte graphique. **Définir le contexte** l'écrit dans les réglages du LLM local, avec sauvegarde et Annuler. Les agents ont besoin d'un contexte assez grand ; quand tu connectes un agent à un modèle LM Studio sans contexte défini, le meilleur est défini dans la même étape.

6.5 Sur ce PC

Liste les fichiers GGUF que le LLM local ne lit pas encore — dans son dossier des modèles, dans ceux des autres LLM locaux, dans `~/Downloads` et dans `~/Models`. **Ajouter** en fait son modèle sans deuxième copie (par un lien), sauf pour Ollama, qui importe sa propre copie.

6.6 Déplacer et supprimer des modèles

Clic droit sur un modèle de llama.cpp, KoboldCpp ou LM Studio › **Déplacer vers…**, ou fais glisser la boîte du modèle sur le dossier d'un autre LLM local. Tu vois d'abord combien est déplacé et quel agent utilise le modèle. Annuler remet les fichiers en place. **Supprimer…** demande d'abord et déplace les fichiers dans la corbeille.

6.7 Un seul dossier des modèles

Réglages › **Dossier des modèles** fixe un dossier pour les modèles de tous les LLM locaux. **Ranger les modèles…** montre à l'avance comment les fichiers seraient classés (`<dossier>/<éditeur>/<modèle>/`), et rien ne change tant que tu n'appuies pas sur **Ranger**. Avec **Les anciens chemins restent valables** (activé par défaut), un lien reste à chaque ancien emplacement. **Annuler le dernier rangement** remet tout en place.

> **ATTENTION** Lors d'un déplacement vers un autre disque, les modèles sont d'abord copiés et les originaux ne sont retirés qu'une fois la copie terminée. Vérifie qu'il y a assez de place libre sur la destination ; sinon, l'app refuse le déplacement.

CHAPITRE 7

Connecter en dessinant

Appuie sur une boîte de la carte et fais glisser jusqu'à une autre pour les connecter. Chaque modification montre, avant d'écrire, exactement quelles lignes de quel fichier changent, fait une sauvegarde et peut être annulée depuis la barre qui suit.

7.1 Agent ↔ LLM local

Fais glisser d'un agent vers un LLM local ou l'inverse. Choisis le modèle et si l'agent l'utilise tout de suite. Pour modifier une connexion, sélectionne son agent (ou clique sur la ligne) : chaque ligne qu'on peut déplacer ou retirer montre un **crochet orange** à son extrémité. Fais glisser le crochet vers un autre LLM local — ou dans le vide pour la retirer ; survoler un crochet indique ce que fait le glisser. Avec un agent sélectionné, un clic sur un modèle d'un LLM local connecté fait de ce modèle celui de l'agent.

Agents pris en charge : Hermes, Pi, Oh My Pi, OpenCode, OpenClaw et DeepSeek Harness.

7.2 Agent ↔ serveur MCP

Fais glisser d'un agent vers un serveur MCP. Les réglages du serveur sont copiés dans le format propre à cet agent. Pris en charge : Claude Code, Codex, Copilot, Crush, Cursor, Gemini, Goose, Grok, Hermes, Muse, Oh My Pi, OpenClaw, OpenCode et Pi. Les fichiers de réglages pouvant contenir des clés sont toujours écrits de façon que toi seul puisses les lire.

7.3 Agent ↔ dossier de skills

Fais glisser d'un agent vers un dossier de skills. Pi et Hermes reçoivent le dossier dans leurs réglages ; tout autre agent reçoit un lien par skill dans son propre dossier de skills. Un dossier qu'un agent lit de lui-même (la plupart lisent `~/.agents/skills`) ne peut pas être retiré à ce seul agent.

7.4 Quand une connexion échoue

Juste après l'écriture d'une connexion, l'app vérifie si elle fonctionnera : si un LLM local est installé et propose le modèle, si un serveur MCP répond, si les liens des skills sont intacts. Une connexion échouée reste sur la carte sous forme de **ligne rouge en pointillés** avec un **!** ; une connexion avec seulement des avertissements est **jaune en pointillés**. Clique sur la ligne ou son **!** pour des conseils :

- **Tester à nouveau** la vérifie encore ; si elle passe, la ligne redevient normale.
- **Réessayer…** ouvre la page de connexion pour une connexion qui n'a jamais été écrite.
- **Supprimer la connexion** la retire.

CHAPITRE 8

Serveurs MCP

Les serveurs MCP donnent des outils supplémentaires aux agents. **+ Serveur MCP** sur la carte ou sous les cartes ouvre la page d'ajout : une liste à gauche, la page du serveur choisi à droite.

8.1 D'où viennent les serveurs

Onglet	Contenu
Recommandés	Les serveurs de référence du projet MCP (Filesystem, Fetch, Git, Memory, Sequential Thinking, Time, Everything) et des serveurs officiels d'entreprises connues (GitHub, Playwright, Chrome DevTools, Brave Search, Context7), regroupés par usage et fixés à leur dernière version.
Registre MCP	Une recherche dans le registre MCP officiel. Chaque résultat nomme son éditeur tel que le registre l'a vérifié. La recherche peut prendre dix secondes ou plus. Les résultats arrivent par 20 ; la ligne sous la liste les fait défiler page par page et montre où tu en es, par exemple **‹ 20 précédents** · 21 – 40 / 200 · **20 suivants ›** (avec */ 40+* quand le registre ne dit pas combien il y en a).
Sur ce PC	Des programmes de cet ordinateur qui proposent un serveur MCP, trouvés en lisant leurs fichiers — rien n'est démarré. On ne regarde dans les AppImages que si tu cliques sur **Regarder dedans…**.
À la main	Une ligne de commande ou une adresse web avec des en-têtes.

Vous ne savez pas de quel serveur vous avez besoin ? Dans l'onglet **Registre MCP**, la carte **Demander à un agent** sous la recherche accepte une demande formulée avec vos mots — par exemple *lire et écrire mes pages Notion*. **Demander** (ou `Ctrl` + `Entrée`) fait transformer par l'agent des vérifications IA cette demande en quelques mots de recherche ; l'application cherche avec eux dans la liste du registre qu'elle a téléchargée, comme la recherche normale, et l'agent choisit parmi les résultats les serveurs qui conviennent, chacun avec la raison. Il ne cherche pas sur le web, ne voit que votre demande et les textes publics du registre, et ne peut choisir que des serveurs vraiment présents dans la liste. Un indicateur animé et le temps écoulé s'affichent pendant ce temps ; après quelques secondes à une minute, les suggestions apparaissent dans la liste des résultats, jusqu'à 10 (**Suggestions** à droite de la ligne de l'agent : 5, 10, 15 ou 20). Une recherche saisie ensuite les remplace.

8.2 Configurer un serveur

La page du serveur dit qui l'a fait et à quel point c'est vérifié, ce qu'il fait (avec des choses que tu peux demander à un agent), comment il tourne — comme programme via `npx`, `uvx` ou Docker, ou comme service web — et ce dont il a besoin, comme un dossier ou une clé API avec un lien pour l'obtenir. En dessous, **Configurer** :

1. Remplis les champs dont le serveur a besoin.

2. Coche les agents qui doivent le recevoir.
3. Clique sur **Tester** pour demander ses outils au serveur.
4. Clique sur **Ajouter** : tous les agents cochés sont écrits d'un coup, avec un aperçu et un Annuler.

> **REMARQUE** Les valeurs secrètes, comme les clés API, sont passées dans l'environnement ou dans des en-têtes, jamais dans une ligne de commande où d'autres utilisateurs de l'ordinateur pourraient les lire. Un serveur qui aurait besoin d'un secret dans sa ligne de commande doit être configuré à la main.

8.3 Gérer les serveurs

ⓘ **Détails** sur la carte d'un serveur MCP montre le fichier de réglages de chaque agent, la commande ou l'adresse et, à la demande, les outils du serveur ; les valeurs des clés ne sont jamais affichées. **Retirer** le retire de tous les agents. Les serveurs restent dans la liste quand plus aucun agent ne les utilise, pour qu'une ligne puisse les reconnecter.

Quand une version plus récente d'une app qui fournit un serveur MCP apparaît dans `~/Applications`, un conseil le signale. **Mettre à jour…** décompresse la nouvelle version, y fait pointer chaque agent et garde l'ancienne sept jours en sauvegarde. **Rechercher des mises à jour** (sous **Sur ce PC**) le fait pour toutes ces apps à la fois après ton clic : il regarde dans les nouvelles versions, propose toutes les mises à jour dans un aperçu avec une sauvegarde et un Annuler, puis teste chaque serveur mis à jour ; un conseil nomme ceux qui ne répondent pas.

CHAPITRE 9

Skills

Une skill est un dossier d'instructions qui apprend à un agent une tâche précise. **+ Skills** ouvre une page organisée comme celle des MCP.

9.1 Trouver des skills

- **Sur ce PC** : des dossiers de skills apportés par des apps et qu'aucun agent ne lit encore.
- **Depuis le web** : une recherche dans l'annuaire skills.sh, sur un autre site avec la même API ou dans un dépôt GitHub de skills. Une skill est récupérée depuis son dépôt GitHub dans un dossier de ton choix, par défaut `~/.agents/skills`.

> **ATTENTION** Les skills peuvent contenir des scripts qu'un agent pourra exécuter plus tard. La page montre le dépôt et liste les scripts avant toute récupération — jette-leur un œil.

9.2 Dossiers de skills

Chaque carte de dossier de skills montre ses skills et les agents qui le lisent. Un dossier partagé par des liens n'apparaît qu'une fois, là où il se trouve vraiment. ⓘ **Détails** liste chaque skill avec sa description et la façon dont chaque agent reçoit le dossier.

CHAPITRE 10

Réglages et aide

10.1 Réglages

L'engrenage (`Ctrl`+`,`) ouvre les réglages. `Retour arrière` ou `Échap` revient en arrière.

Réglage	Ce qu'il fait
Langue	La langue du système ou English, Deutsch, Français, Español, Italiano, Português, Nederlands, Polski.
Apparence OMARCHY	Les couleurs du thème Omarchy ou l'apparence Adwaita de GNOME avec le mode clair/sombre et la couleur d'accent du système.
Animations de la carte	Une lumière parcourt les lignes de la carte : **Agents et autres programmes** (la ligne d'un agent qui travaille et la ligne d'un LLM local vers un modèle qu'un autre programme utilise), **Seulement mes agents** ou **Désactivé**. Une petite carte animée sous le réglage montre ce que chaque choix anime.
Encadré des statistiques d'utilisation	Les nombres le long de la ligne d'un agent au travail (voir Travailler avec la carte) : activé ou non, lesquels de **Jetons par seconde**, **Taille du contexte** et **Contexte utilisé** il affiche, et sa **Taille du texte** (7 pt par défaut).
Sous-agents	Pour la page Sous-agents : combien de temps restent les sous-agents terminés (**30 minutes** par défaut ; 10, 30, 60 ou 120) et combien au plus (**10** par défaut ; 5, 10, 20 ou 40). Ceux qui travaillent sont toujours affichés.
Coûts	**Devise** : dollar américain, ou euro au cours de référence quotidien de la Banque centrale européenne (récupéré une fois par jour sur ecb.europa.eu). **Limite par jour** pour un agent payé au token (**100** par défaut ; de 10 à 500). **Jaune à partir de** et **Rouge à partir de** : la part utilisée à partir de laquelle une indication de coût ou une batterie devient jaune (75 % par défaut) et rouge (90 %). **Avertir au rouge** (activé par défaut) : une notification de bureau quand un agent passe au rouge — une fois par limite jusqu'à sa remise à zéro, ou une fois par jour pour un coût.
Onglets de terminal au-dessus de la carte	Un onglet par terminal d'agent dans la fenêtre.
Coins	Comme les fenêtres, arrondis ou carrés.
Démarrer les LLM locaux avec leurs agents	Le veilleur d'arrière-plan de Démarrer et arrêter avec les agents.
Dans le lanceur	L'entrée du lanceur ; désactivé la retire.
Les agents démarrent dans	Le dossier dans lequel démarrent les agents.

Réglage	Ce qu'il fait
Fonctions IA : **Rechercher les nouveaux agents**	Demander à un agent installé où un agent inconnu range ses réglages (activé par défaut).
Fonctions IA : **Vérifications IA quotidiennes**	Une fois par jour, un agent cherche sur le web de nouvelles versions de tes serveurs MCP (désactivé par défaut ; cela utilise les jetons de l'agent). Voir Smart Check.
Fonctions IA : **Agent pour les vérifications IA**	Claude Code ou Codex : fait les vérifications et recherches IA et prend les tâches **Le faire avec …** et **Corriger avec …**.
Dossier des modèles	Un dossier pour tous les modèles, avec **Ranger les modèles…**.
Modules ASTERISM	**Chercher des modules en ligne**, la page des modules et **Configuration**.
À propos	La version, l'auteur et la licence ; **Site web** ouvre le projet sur GitHub et **Signaler un problème** sa liste de tickets.

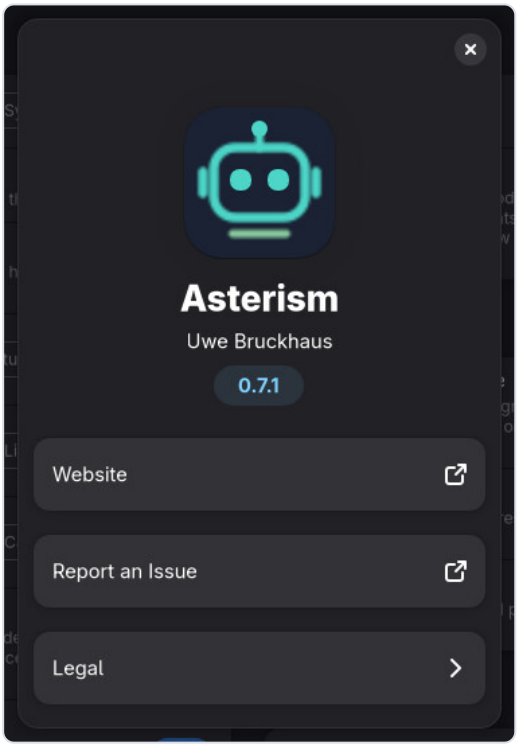


Figure 11 · Réglages › À propos.

10.2 Aide

? (F1) ouvre l'aide : **Ouvrir le manuel** ouvre ce manuel (l'édition livrée avec ta version, dans ta langue) dans ta visionneuse PDF ; **Espace disque** (ci-dessous) ; une animation sur la configuration en dessinant ; et LLM Monitor, un plugin de barre compagnon qui montre ce que fait un LLM local (une

petite animation de son entrée dans la barre et de son panneau). Ces deux animations sont dessinées dans les couleurs de ton thème et suivent aussitôt un changement de thème. La légende ⓘ de l'en-tête explique chaque point, ligne et marque de la carte.

Le **i** plein de l'en-tête ouvre **Informations** : des cartes pour les autres plugins de l'auteur, lues sur l'Omarchy Plugin Marketplace à l'ouverture de la page, au plus une fois par heure (sans réseau, la liste que connaît cette version). Un clic sur une carte ouvre la page du plugin sur le marketplace.

10.3 Espace disque

Espace disque : agents, LLM locaux et modèles montre combien de place prend ta configuration IA et où. Il est mesuré en arrière-plan à chaque ouverture de l'aide (de nouveau après dix minutes ou avec **Mesurer à nouveau**) et seulement lu — rien n'est supprimé d'ici. Le titre montre toujours les totaux sur une ligne (*Agents · LLM locaux · Modèles · Au total*) ; un clic replie la section. **Disques**, **Agents**, **LLM locaux** et **Dossiers de modèles** commencent repliés, chacun avec son total et les éléments les plus gros ; un clic les ouvre.

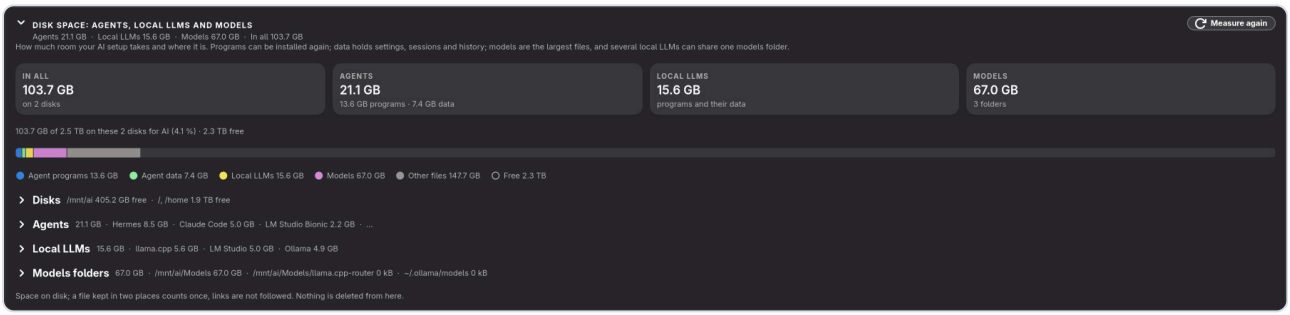


Figure 12 · Les totaux, la part de l'IA dans tout l'espace disque et les sections repliées avec leurs totaux.

- **Tuiles** : tout ensemble, puis programmes des agents, données des agents (réglages, sessions, historique), LLM locaux (programmes et leurs propres données) et modèles.
- **Barre** : tout l'espace des disques où se trouve la configuration IA. Les parties colorées sont la configuration IA, le gris est tout le reste sur ces disques et le reste vide est libre ; la ligne au-dessus donne la part de l'IA en pourcentage. Survole la barre pour les tailles exactes.
- **Disques** : chaque disque avec ce que la configuration IA y occupe et ce qui reste libre.
- **Agents**, **LLM locaux**, **Dossiers de modèles** : une ligne chacun, les plus gros d'abord. Ouvre une ligne pour voir chaque dossier avec sa taille et son nombre de fichiers, et l'ouvrir dans le gestionnaire de fichiers. Un dossier de programme dit combien de versions sont gardées ; pour un agent avec plusieurs anciennes versions dans mise, une astuce mentionne `mise prune`, qui retire celles qui ne servent pas. Un dossier de modèles dit quels LLM locaux l'utilisent, ou qu'il ne contient que des liens vers des modèles d'un autre dossier.

> **ASTUCE** Les programmes peuvent être réinstallés et les modèles retéléchargés ; les données des agents contiennent tes sessions et tes réglages. Quand la place manque, regarde d'abord les anciennes versions de programmes et les modèles que tu n'utilises plus.

Les tailles sont de l'espace disque : un fichier conservé à deux endroits compte une fois, et les liens ne sont pas suivis.

CHAPITRE 11

Smart Check

Smart Check laisse un agent en ligne installé — Claude Code ou Codex — examiner la configuration IA de ce PC, trouver des mises à jour pour tes serveurs MCP et faire des corrections pour toi, et montre ce que cela a coûté en jetons. Les deux étincelles de l'en-tête l'ouvrent ; elles n'apparaissent que si l'un de ces agents est installé. Réglages › **Agent pour les vérifications IA** choisit lequel.

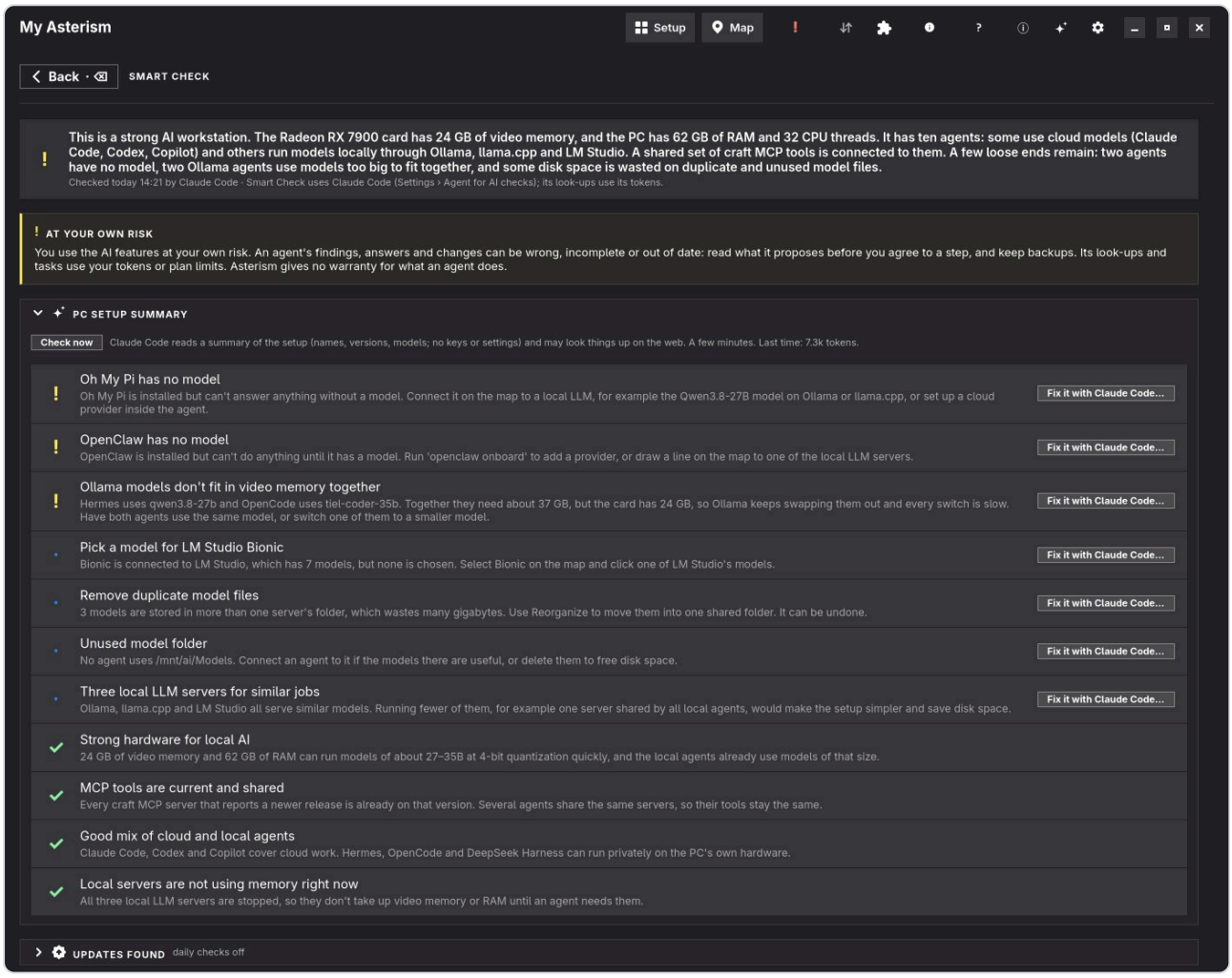


Figure 13 · Smart Check après une vérification par Claude Code : problèmes (!), astuces (·) et ce qui va bien (✓).

> **À VOS PROPRES RISQUES** Les constats, réponses et modifications d'un agent peuvent être faux, incomplets ou dépassés. Lis ce qu'il propose avant d'accepter une étape, et garde des sauvegardes. Ses recherches et tâches utilisent tes jetons ou les limites de ton forfait. L'app ne donne aucune garantie sur ce que fait un agent. La même note figure en tête de Réglages › **Fonctions IA**.

11.1 Résumé de la configuration du PC

Vérifier maintenant donne à l'agent un court texte sur la configuration — la carte graphique et la mémoire, les agents avec leurs modèles, les LLM locaux, les serveurs MCP avec leurs versions, les skills et ce qu'ont trouvé les Problèmes ouverts — et le laisse chercher sur le web. Il ne contient ni clés, ni en-têtes, ni valeurs de réglages. Cela prend une minute ou plus en arrière-plan ; la ligne sous le bouton dit combien de jetons la dernière vérification a utilisés. La réponse est un court résumé et une liste de constats : **!** problèmes, • astuces et ✓ ce qui va bien. Les réponses qui ne tiennent pas sont écartées.

Corriger avec … à côté d'un problème ou d'une astuce ouvre l'agent dans un onglet de terminal de la fenêtre, avec ce constat comme tâche : il regarde si c'est bien le cas, explique et te demande avant chaque étape qui change quelque chose. Passer des serveurs MCP à une nouvelle version est laissé au bouton **Mettre à jour** de l'app (ci-dessous), qui modifie les réglages de tous les agents à la fois, avec une sauvegarde et Annuler.

11.2 Mises à jour trouvées

Les nouvelles versions de tes serveurs MCP, de deux sources :

- **Les vérifications IA quotidiennes** (Réglages › **Vérifications IA quotidiennes**, désactivé par défaut) : une fois par jour, l'agent cherche sur les pages de versions officielles des versions plus récentes des serveurs dont l'app connaît la version — les apps décompressées dans `~/Applications/<app>-<version>/` et les paquets npm ou PyPI fixés à une version. **Vérifier maintenant** sur cette carte demande tout de suite, même quand les vérifications quotidiennes sont désactivées. Chaque trouvaille a **Page de la version** et **Le faire avec …** : l'agent télécharge la nouvelle AppImage dans un onglet de terminal, en demandant avant chaque étape.
- **La recherche de l'app elle-même** : une version plus récente est déjà sur ce PC, mais un agent démarre encore l'ancienne (la même chose que la mise à jour sur la carte du serveur dans Configuration).

Une fois la nouvelle version sur ce PC, sa ligne le dit et propose **L'utiliser…** : l'aperçu de la mise à jour montre quels agents passent à elle ; **Mettre à jour** les modifie tous, garde l'ancienne version avec la sauvegarde et teste ensuite chaque serveur. Un serveur dont tous les agents démarrent la nouvelle version disparaît de la liste.

> **ASTUCE** Après une action lancée ici — une mise à jour, ou une tâche dans un onglet de terminal — **Retour** te ramène à Smart Check, à jour.

11.3 Statistiques de jetons

Les jetons utilisés aujourd'hui et sur les 30 derniers jours, un graphique par jour, et les totaux par agent et par plugin (les autres plugins Omarchy qui utilisent le même assistant IA apparaissent ici aussi). **Valeur API** est ce que ces exécutions coûteraient au prix de l'API, et la ligne en dessous dit si ton abonnement le couvre ou si c'est facturé sur une clé d'API. Une tâche dans un onglet de terminal compte dans la session propre de son agent ; la page de détails de l'agent montre ces jetons.

En dessous, une carte par type de tâche — la vérification de la configuration, les mises à jour de serveurs MCP, la recherche de nouveaux agents, les corrections — avec ses exécutions, ses jetons et les derniers résultats.

CHAPITRE 12

Bilan de santé et de sécurité

Un agent de programmation lit et écrit des fichiers et exécute des commandes sur ton ordinateur. Ce qu'il peut faire *sans te demander* se trouve dans ses propres réglages, avec des mots différents pour chaque agent. Le **Bilan de santé et de sécurité** rassemble tout cela : sur la page de détails de chaque agent et de chaque LLM local, quatre cartes montrent s'il fonctionne, ce qu'il peut faire, ce que cela signifie en mots simples et ce qui réduit le risque.

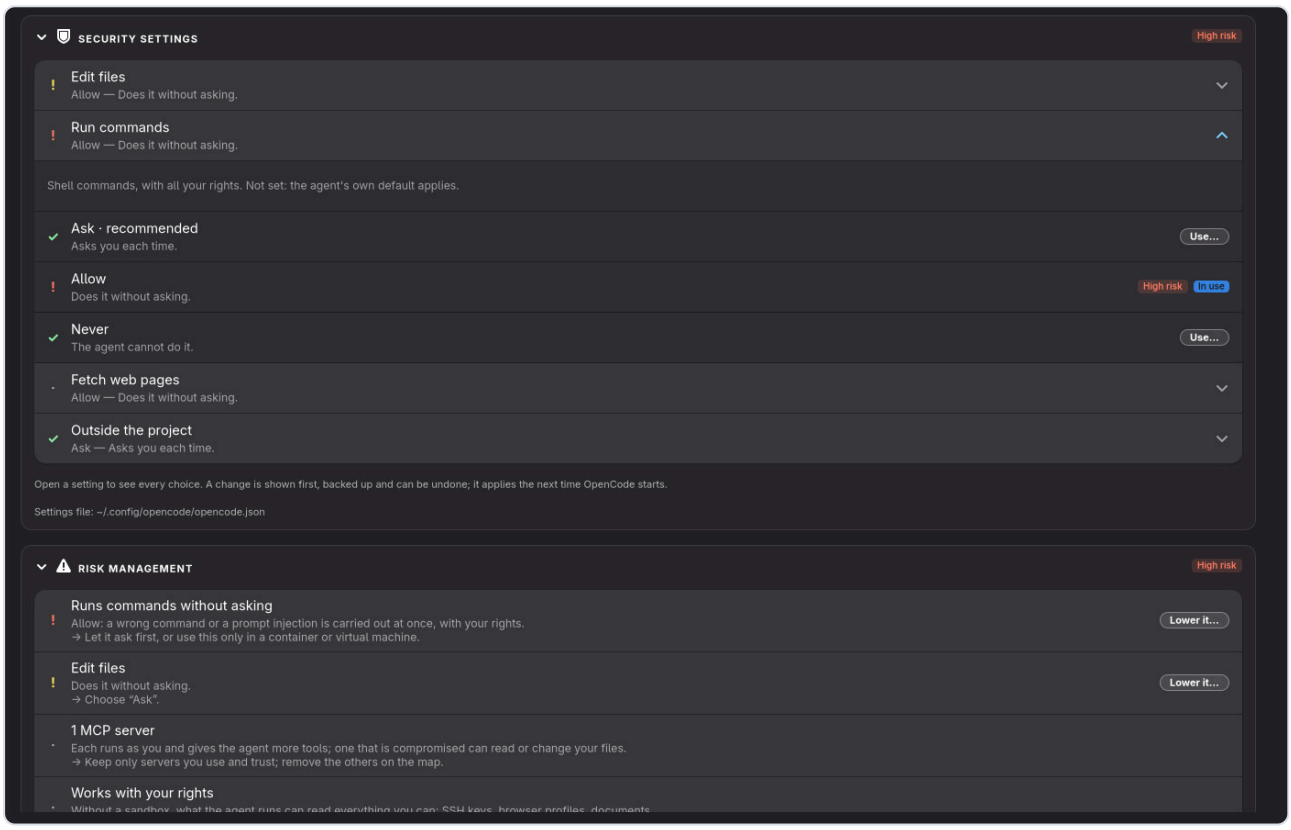


Figure 14 · Les réglages de sécurité d'OpenCode avec **Exécuter des commandes** ouvert, et le début de sa gestion des risques.

12.1 Lire les cartes

Les badges des cartes repliées le résument : **En bonne santé**, **À vérifier** ou **Non configuré** sur État, et **Risque faible**, **Risque moyen** ou **Risque élevé** sur Réglages de sécurité et Gestion des risques. Ouvre une carte pour le détail :

Carte	Ce qu'elle montre
État	Chaque vérification de la configuration avec un ✓, ou ce qu'il faut faire ensuite : installé, connecté, son modèle et son LLM local, serveurs MCP et skills.
Réglages de sécurité	Les réglages de l'agent qui décident de ce qu'il peut faire seul, chacun avec sa valeur actuelle et une phrase sur ce qu'elle signifie. Le signe devant indique le risque : ✓ sûr, · faible, un **!** jaune moyen, un **!** rouge élevé.

Carte	Ce qu'elle montre
Gestion des risques	Ce que les réglages signifient ensemble, le plus sérieux d'abord, chacun avec la raison et → ce qui le réduit. Les protections en place apparaissent avec un ✓.
Ce que veulent dire les mots	Sandbox, approbation, règles d'autorisation et d'interdiction et les autres mots des cartes, expliqués (voir Les mots expliqués).

Les règles d'autorisation et d'interdiction et les dossiers de confiance de l'agent (par exemple la liste `permissions.allow` de Claude Code) apparaissent sous les réglages, une règle par ligne.

12.2 Modifier un réglage

1. Clique sur un réglage pour l'ouvrir. Tu vois ce qu'il contrôle et chaque choix avec son risque ; celui qui est utilisé porte la marque **Utilisé**, le choix plus sûr **recommandé**.
2. Clique sur **Utiliser …** à côté d'un choix ou sur **Réduire …** à côté d'un risque. Une page montre la modification en mots (avant → après), le risque avant et après, et **Ce qui change** : les lignes du fichier de réglages de l'agent.
3. Clique sur **Modifier**. Le fichier est sauvegardé et écrit ; une notification propose **Annuler** pendant 30 secondes. L'agent utilise le nouveau réglage à son prochain démarrage.

> **REMARQUE** Seule cette clé change ; les commentaires et tout le reste du fichier restent tels quels. Un réglage qui demande plus d'une ligne dans un fichier — comme Hermes qui exécute ses commandes dans Docker — est montré et expliqué, avec l'endroit où le modifier, mais n'est pas modifié d'ici.

12.3 Ce qui est vérifié pour chaque agent

Agent	Réglages affichés et modifiables	Fichier
Claude Code	Mode de permissions (Demander d'abord, Planifier seulement, Seulement l'autorisé, Modifier sans demander, Revue automatique, Contourner toutes les vérifications), sandbox pour les commandes, verrouiller le mode contournement ; règles autoriser/demander/interdire	`~/.claude/settings.json`
Codex	Quand Codex demande, sandbox (lecture seule, dossier du projet, aucune), réseau dans la sandbox ; dossiers de confiance	`~/.codex/config.toml`
GitHub Copilot	Mode de permissions (Demander d'abord, Revue automatique, Tout autoriser) ; dossiers de confiance, sites sans demander	`~/.copilot/settings.json`
Cursor CLI	Approbation, sandbox, réseau dans la sandbox, recherche web sans demander ; règles autoriser/interdire	`~/.config/cursor/cli-config.json`

Agent	Réglages affichés et modifiables	Fichier
Hermes	Commandes dangereuses (toujours demander, revue automatique, ne jamais demander), masquer les secrets ; où tournent les commandes (affiché seulement)	`~/.hermes/config.yaml`
OpenCode	Demander, autoriser ou jamais pour modifier des fichiers, exécuter des commandes, récupérer des pages web et travailler hors du projet	`~/.config/opencode/opencode.json`
Pi	Ses outils : lire, écrire et exécuter des commandes, ou seulement lire. Pi ne demande jamais par conception, ses outils sont donc ses permissions.	`~/.pi/agent/settings.json`

Pour les autres agents, la carte montre leur état et leurs serveurs MCP et dit où regarder dans les propres réglages de l'agent. Pour un **LLM local**, elle montre qui peut l'atteindre : ce PC seulement (`127.0.0.1` , comme il se doit) ou n'importe quel appareil de ton réseau.

12.4 Maîtriser le risque

Le niveau de risque est le point le plus sérieux que trouve la carte :

- **Risque élevé** — il peut faire des choses sérieuses sans demander : exécuter n'importe quelle commande, ou travailler sans sandbox et sans questions. Acceptable dans un conteneur jetable ou une machine virtuelle ; sur ton propre PC, fais-le demander d'abord.
- **Risque moyen** — certaines choses se font sans demander, comme modifier des fichiers du projet, ou une revue automatique décide à ta place.
- **Risque faible** — il demande avant tout ce qui compte, ou ne peut pas le faire du tout.

Bonnes habitudes, quels que soient les réglages :

- **Demander d'abord pour les commandes** sur ton propre ordinateur ; autorise des commandes isolées et anodines (comme `git status`) avec une règle d'autorisation plutôt que tout le shell.
- **Active la sandbox** là où l'agent en a une (Claude Code, Codex, Cursor CLI) et laisse le réseau désactivé à l'intérieur tant qu'aucune tâche n'en a besoin.
- **Démarre les agents dans le dossier du projet**, pas dans ton dossier personnel, et garde clés et mots de passe à l'écart.
- **Peu de serveurs MCP** : seulement ceux que tu utilises et en qui tu as confiance. Chacun tourne avec tes droits.
- **Le travail sans surveillance** (mode contournement, « YOLO », Pi avec tous les outils) se fait dans un conteneur ou une machine virtuelle.
- **Les LLM locaux sur `127.0.0.1`** : leurs serveurs n'ont pas de mot de passe.

> **ATTENTION** Une revue automatique (le mode *auto* de Claude Code, *assisted* de Copilot, *auto-review* de Cursor, *smart* de Hermes) est une seconde IA qui juge chaque étape. Elle épargne des questions, mais c'est un jugement, pas un mur.

12.5 Les mots expliqués

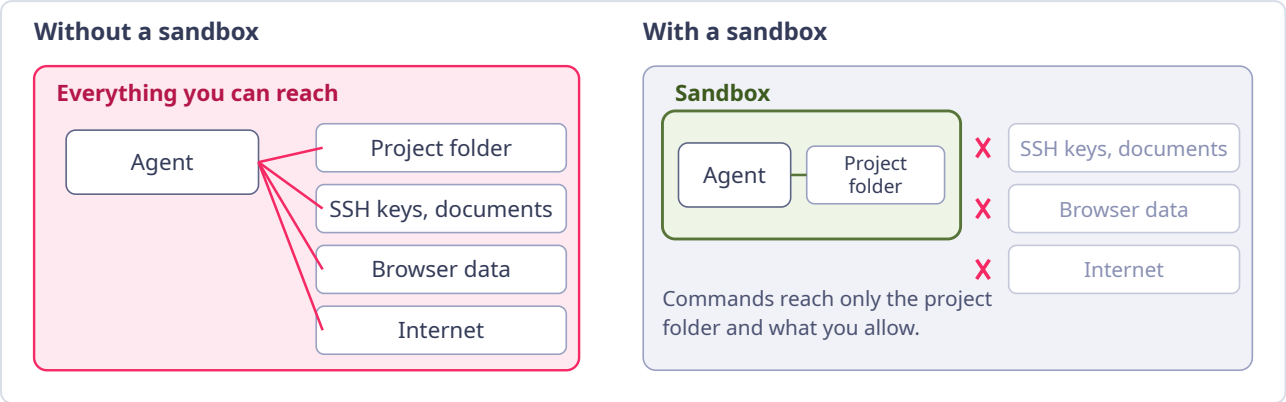


Figure 15 · Ce que change une sandbox : les commandes n'atteignent que le dossier du projet et ce que tu autorises.

Sandbox	Un espace isolé où tournent les commandes de l'agent. À l'intérieur, une commande ne peut toucher que le dossier du projet (et ce que tu autorises) — pas le reste de ton dossier personnel, tes clés SSH ou les données du navigateur, et souvent pas internet. Sans sandbox, une commande de l'agent peut faire tout ce que tu peux faire.
Approbation (demander d'abord)	Avant que l'agent ne modifie un fichier ou n'exécute une commande, il montre ce qu'il veut faire et attend ton oui ou ton non. Moins il peut faire sans demander, plus tu gardes le contrôle — et plus souvent il te demande.
Règles d'autorisation et d'interdiction	Des listes de commandes ou d'outils que l'agent peut utiliser sans demander (autoriser) ou jamais (interdire). Interdire l'emporte sur autoriser. Une règle qui autorise n'importe quelle commande shell désactive l'approbation pour le shell.
Dossier de confiance	Un dossier dans lequel tu as dit que l'agent peut travailler. Là, il utilise aussi les propres instructions et réglages du dossier, qui peuvent venir de quelqu'un d'autre quand tu clones un projet.
Accès au réseau	Si les commandes peuvent atteindre internet. Du code qui lit tes fichiers et peut envoyer des données pourrait les divulguer ; installer des paquets a aussi besoin du réseau.
Contournement (« YOLO »)	Tout sans demander et sans limites. Seulement pour une VM ou un conteneur jetable.
Revue automatique	Un second modèle d'IA laisse passer les actions anodines et demande pour le reste.
Injection de prompt	Des instructions cachées dans une page web, un fichier ou la réponse d'un outil, qui essaient de faire faire à l'agent quelque chose que tu n'as pas demandé. Demander d'abord, une sandbox et pas de réseau les arrêtent.
Masquer les secrets	Un texte qui ressemble à un mot de passe, un jeton ou une clé est masqué avant que le modèle ne le voie ou qu'il n'atterrisse dans un journal.
Outils	Ce que l'agent peut faire tout court : lire, écrire, exécuter des commandes, chercher sur le web. Un outil qu'il n'a pas ne peut pas être détourné.
Serveur MCP	Un petit programme qui donne des outils supplémentaires à l'agent. Il tourne en tant que toi, avec tes droits.
127.0.0.1 (localhost)	Une adresse que seuls les programmes de ce PC atteignent. Un serveur sur `0.0.0.0` est accessible depuis n'importe quel appareil de ton réseau.

CHAPITRE 13

Modules

Tout ce qu'Asterism comprend d'un agent ou d'un fournisseur de LLM est un **adaptateur** : comment il est trouvé et installé, ses sessions, ses réglages de sécurité, ses serveurs MCP et ses skills. Dans Asterism, chaque adaptateur est un **module** : les paquets les apportent tous, signés et désactivés, et tu actives ceux que tu utilises. Les modules apportent aussi de nouveaux agents, de nouveaux fournisseurs et des fonctions au-delà des chiffres, comme **Insights** (sujets des sessions et un résumé par Claude). Le plugin AI Agents Map a les adaptateurs intégrés et n'a pas de modules.

13.1 Configurer Asterism

Un Asterism neuf s'ouvre sur sa **page d'accueil**. Chaque agent et LLM local qu'Asterism connaît est une carte avec sa propre icône, en trois groupes : **Agents IA**, **LLM locaux** et **Fournisseurs en ligne** (les noms d'Anthropic, d'OpenAI et des autres sur la carte). Chaque carte dit si le programme est là :

- **Installé** (vert) — trouvé de la même façon que la carte le trouve ; la carte est déjà cochée ;
- **Utilisé auparavant** — pas installé maintenant, mais ses réglages sont toujours là ;
- **Non installé** — coche-le pour l'installer ensuite depuis sa carte.

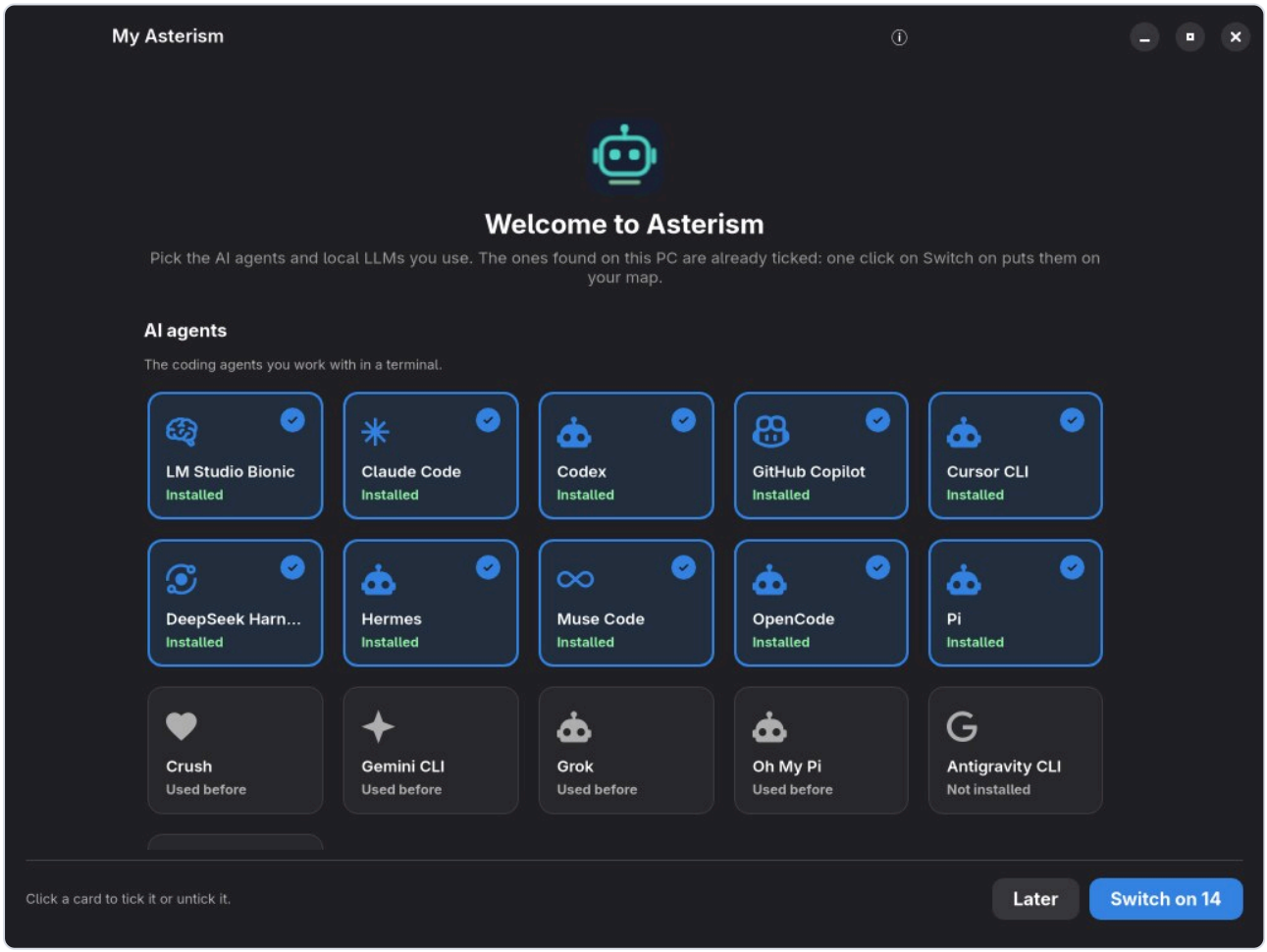


Figure 16 · La page d'accueil : les programmes trouvés viennent en premier et sont cochés.

Clique sur une carte (ou appuie sur `Espace` dessus) pour la cocher ou la décocher ; une carte cochée a une bordure bleue et une coche. **Activer** (avec le nombre de cartes cochées) les charge et montre la carte ; si tu as coché des programmes qui ne sont pas installés, **Configuration** s'ouvre : ··· sur leurs cartes › **Installer…**. **Plus tard** démarre avec une carte vide. Tant que rien n'est activé, Asterism se rouvre sur la page d'accueil à chaque démarrage. Ensuite, les mêmes cartes se trouvent dans **Configuration** (le bouton de l'en-tête), où agents et fournisseurs s'activent et se désactivent à tout moment.

13.2 La page des modules

Ouvre-la avec le bouton pièce de puzzle de l'en-tête, depuis Réglages › **Modules** ou depuis n'importe quel conseil « module disponible ». Elle montre, dans des cartes de groupe repliables qui commencent ouvertes :

- **Suggérés pour ce PC** — un module dont l'agent ou le fournisseur est installé ici mais pas encore affiché ;
- **Modules installés** — activé, désactivé ou *non chargé* avec la raison (par exemple un fichier modifié) ; les agents et les fournisseurs séparés par une ligne ;
- **Disponibles** — le reste des modules du catalogue ;
- **Intégré à Asterism** — les agents et fournisseurs qu'aucun module installé n'apporte (quand tu lances Asterism depuis son dossier source sans modules).

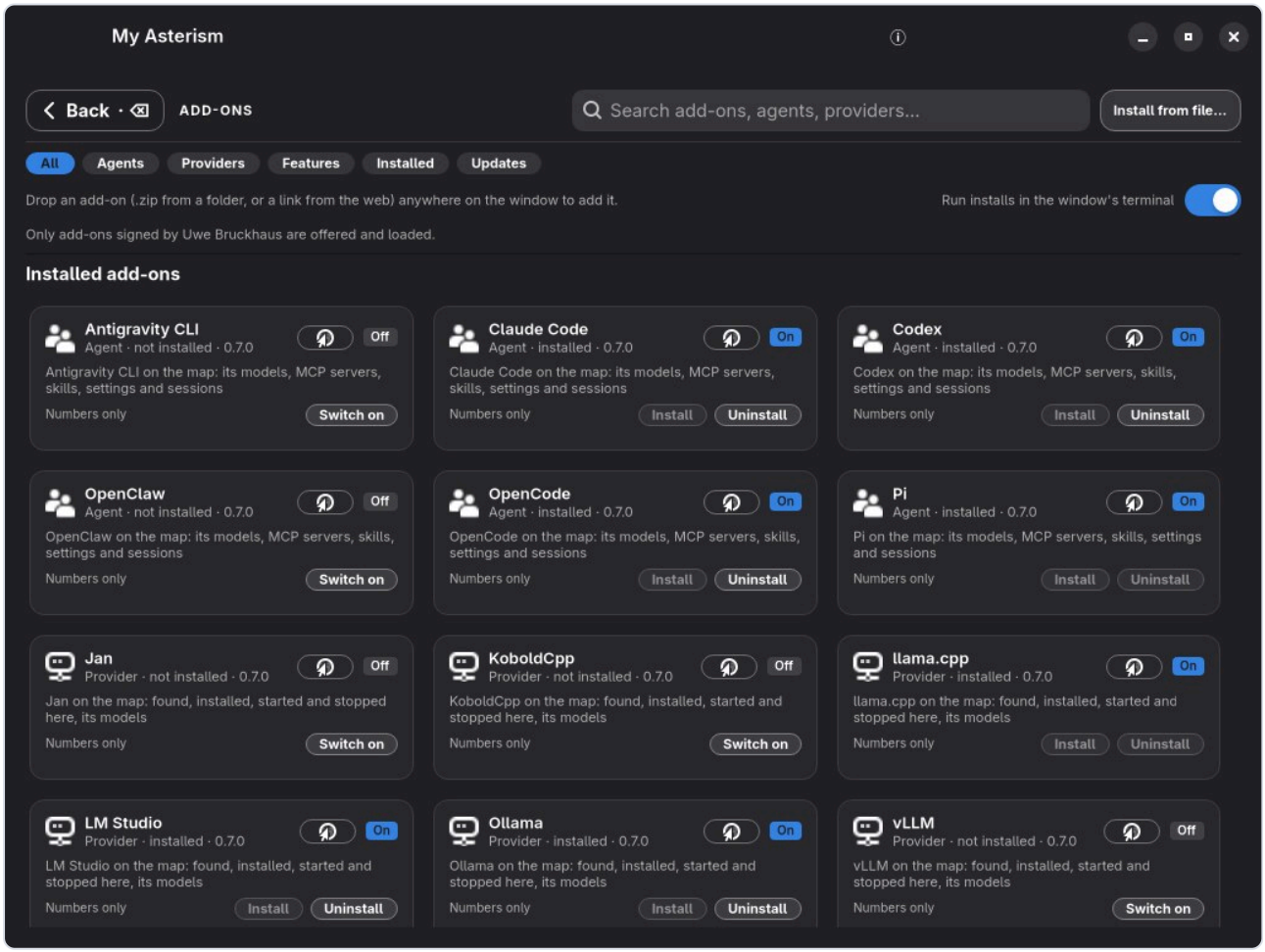


Figure 17 · Modules installés : activés ou désactivés, avec Installer et Désinstaller pour le programme.

La carte d'un agent ou d'un LLM local activé a **Installer** et **Désinstaller** pour le programme lui-même (celui qui ne s'applique pas est grisé), et chaque carte avec une page d'accueil a un bouton globe qui l'ouvre dans ton navigateur.

La recherche trouve les noms, les descriptions et ce qu'un module lit ; les filtres au-dessus limitent la liste aux agents, fournisseurs, fonctions, aux installés ou à ceux qui ont une mise à jour. Clique sur une carte pour ses détails : ce qu'il lit, ce qu'il envoie, ce qu'il peut faire et son niveau de confidentialité.

13.3 Installer, activer et désactiver, mettre à jour, retirer

Installer (ou **Obtenir le module**) montre d'abord ce que le module lit, envoie et peut faire ; ce n'est qu'après ton accord qu'il est téléchargé, vérifié et activé. **Désactiver** et **Activer** agissent aussitôt, sans redémarrage : ses agents, fournisseurs, sections et cartes apparaissent ou disparaissent partout. **Mettre à jour** le remplace par la version plus récente du catalogue après vérification de la nouvelle ; **Retirer** le déplace dans la corbeille. **Installer depuis un fichier…** prend un zip de module que tu as téléchargé toi-même ; les mêmes vérifications s'appliquent.

> **SEULEMENT DE L'AUTEUR.** Asterism ne charge que des modules signés par Uwe Bruckhaus (empreinte de la clé `SHA256:e5cC634Ke3Our5LAeSc88qs22NKZEHTncBW5Ig3amWQ`). Avant que quoi que ce soit d'un module ne tourne, la signature est vérifiée avec `ssh-keygen` d'OpenSSH, chaque fichier est comparé à sa somme SHA-256, et aucun fichier ne doit avoir été ajouté. Un téléchargement doit en plus correspondre à la somme SHA-256 du catalogue signé.

13.4 Niveaux de confidentialité

Niveau	Ce que le module peut lire ou envoyer
Chiffres uniquement	Comptes, durées, jetons, lignes modifiées, coût — ce qu'Asterism montre sans modules.
Titres de session	Les titres qu'un agent a lui-même donnés à ses sessions (jamais les conversations).
Titres et un résumé par Claude	Ces titres et chiffres, envoyés à ton propre Claude Code — seulement sur un clic, après un aperçu de ce qui est exactement envoyé.

Un module ne peut jamais faire plus que ce que permet son niveau. Ajouter un agent ou un fournisseur, ou installer un programme, sont des permissions séparées que sa carte nomme.

13.5 Types de modules

Type	Ce qu'il apporte
Agent	Un agent de programmation : où il range ses réglages, ses sessions, ses serveurs MCP et ses skills, comment il s'installe et comment il se connecte. Asterism construit tout le reste à partir de cela — la carte dans Configuration, sa boîte sur la carte, Utilisation et jetons, Projets, État et Sécurité.

Type	Ce qu'il apporte
Fournisseur	Un programme de LLM local ou les fournisseurs en ligne : comment il est trouvé et installé et comment les agents l'atteignent. Asterism construit sa carte, sa boîte sur la carte et sa page de détails — y compris **Utilisation et jetons** : ce que tes agents ont envoyé par lui, d'après leurs propres chiffres de session.
Extension	Plus d'informations sur un agent ou un fournisseur qu'apporte un autre module, sans le remplacer — par exemple son abonnement et ses limites, ou une carte sur sa page de détails. Elle ne remplit que ce que le module de l'agent laisse vide ; la page du module dit ce qu'elle ajoute et ce qui n'en est pas utilisé.
Fonction	Quelque chose qui concerne plusieurs agents, comme Insights (la carte Sujets).

La section d'un module sur la page de détails d'un agent ou d'un LLM local va dans la carte qui la concerne : Utilisation et jetons, État, Sujets (agents seulement) ou une carte à son nom.

Un module d'agent décrit son agent sous forme de données (un fichier nommé `agent.toml`) ; la plupart n'apportent aucun code, et la page du module le dit : *N'apporte pas de code* ou, pour les quelques agents qui en ont besoin (par exemple leur propre base de sessions), *Des données sur l'agent, et un peu de code pour ce que les données ne peuvent pas dire*.

13.6 Un nouvel agent sur ton ordinateur

Quand tu installes un agent qu'Asterism ne connaît pas encore, il regarde dans le catalogue (Réglages › **Chercher des modules en ligne**, activé par défaut : la liste est lue sur GitHub toutes les quelques heures et seule la requête est envoyée). Si un module connaît cet agent, **Mises à jour et suggestions** (le **!** de l'en-tête) le dit (*« Kilo Code est sur ce PC mais pas sur la carte »*), et une carte apparaît sous **Suggérés pour ce PC** ; un clic l'installe. S'il n'y a pas encore de module, les détails de l'agent le disent, avec un lien pour en demander un.

13.7 Créer un module

Un nouveau LLM local ou un nouvel agent n'a besoin que d'un module : Asterism construit sa carte, sa boîte et ses lignes sur la carte, ses détails, la page Modèles et la page Sécurité à partir de ce que le module en dit. Si tu sais un peu programmer, pars des modèles du dépôt d'Asterism (`templates/`, expliqués dans `docs/addons.md`) : un fichier pour un LLM local, un fichier de faits pour un agent. Envoie le module à `github.com/uBruckhaus/asterism-addons` ; une fois relu et signé, il apparaît pour tout le monde sous **Disponibles**.

CHAPITRE 14

Confidentialité et sécurité

- **Aucun droit d'administrateur pour l'app.** Elle ne tourne jamais en root et ne voit jamais de mot de passe. Installer un agent fourni comme paquet du système (comme Hermes ou OpenClaw) lance le gestionnaire de paquets du système avec `sudo` dans un terminal que tu vois, après t'avoir montré la commande.
- **Aucun identifiant lu.** Elle lit les réglages des agents, mais jamais leurs fichiers d'identifiants. Qu'un agent soit connecté se voit à l'existence d'un tel fichier. Les valeurs des clés et des en-têtes ne sont jamais affichées.
- **Aucune conversation lue.** Les chiffres d'utilisation ne sont lus que comme des nombres et des durées. La seule exception est le suivi d'une instance *En arrière-plan* (Terminaux…), sur ton clic, pour cette instance seulement ; rien n'en est conservé.
- **Le réseau seulement quand tu le demandes** — chercher ou installer des modèles, des serveurs MCP ou des skills, tester un serveur —, plus une vérification des mises à jour toutes les quelques heures.
- **Téléchargements vérifiés.** Les programmes ne sont installés que si le téléchargement correspond à la somme publiée par leur version.
- **Aperçu et Annuler.** Chaque modification des réglages d'un autre programme est montrée avant, sauvegardée et peut être annulée. Les réglages pouvant contenir des clés sont écrits lisibles par toi seul.
- **Encadré.** Chaque programme tourne sans shell, et chaque réponse est lue avec une limite de taille et de temps.
- **Les fonctions IA seulement quand tu les choisis.** Les recherches de Smart Check se font sur un clic, les vérifications IA quotidiennes seulement quand elles sont activées. L'agent reçoit un court résumé (noms, versions, modèles), jamais de clés ni de valeurs de réglages, transmis d'une façon que les autres utilisateurs de l'ordinateur ne peuvent pas lire ; ses réponses sont vérifiées avant d'être affichées et ne sont jamais exécutées. Les tâches tournent dans un terminal que tu vois, et l'agent demande avant chaque étape. Tu les utilises à tes propres risques.

Le README de chaque dépôt liste en entier ce qui est lu, démarré et écrit.

CHAPITRE 15

Dépannage

Problème	Que faire
La carte est à l'étroit ou coupée.	La fenêtre fait moins d'environ 1300 pixels. Élargis la fenêtre, ou fais un zoom arrière (`Ctrl`+`-`).
Une boîte que j'attends n'est pas sur la carte.	La carte ne montre que ce qui est configuré : regarde dans **Configuration** (un module désactivé, un serveur MCP qu'aucun agent n'utilise) et les interrupteurs **Afficher**.
Lancer ici manque ou ne fait rien.	Installe VTE pour GTK 4 (`vte4`), ou utilise **Lancer dans un terminal**.
Une ligne est rouge en pointillés.	Clique dessus pour des conseils, corrige ce qu'ils indiquent, puis **Tester à nouveau**. Voir Quand une connexion échoue.
Le LLM local ne démarre pas avec l'agent.	Vérifie Réglages › **Démarrer les LLM locaux avec leurs agents** et que l'agent est connecté sur la carte. `systemctl --user status ai-agents-map-providers` montre le veilleur.
La lumière colorée d'un LLM local vers son modèle n'apparaît pas.	Elle n'apparaît que pendant qu'un programme absent de la carte génère avec ce LLM, et s'éteint environ dix secondes après sa dernière requête. Vérifie que la carte est visible (une fenêtre large), que le programme envoie vraiment quelque chose et que Réglages › **Animations de la carte** est sur **Agents et autres programmes**.
L'agent répond mal ou s'interrompt avec un modèle local.	Le contexte est peut-être trop petit. Ouvre le modèle dans **Modèles… › Installés** et utilise **Définir le contexte** avec la meilleure valeur pour ta carte.
Un téléchargement s'est arrêté.	Après une coupure de connexion, il reprend tout seul. Après un redémarrage, utilise **Reprendre** dans la bande du haut.
Je veux annuler une modification antérieure.	Les sauvegardes sont dans `~/.local/state/ai-agents-map/backups/` .

ANNEXE

Référence

A.1 Raccourcis clavier

`F1`	Aide
`Ctrl`+`,`	Réglages
`F5`	Actualiser (la carte et Configuration)
`Ctrl`+`+` / `Ctrl`+`-` / `Ctrl`+`0`	Agrandir la carte, la réduire, revenir à 100 %
`Ctrl`+molette	Zoomer sur la carte
`Maj`+clic	Ajouter à la sélection ou en retirer
`Échap`	Effacer la sélection ; revenir d'une page
`Retour arrière`	Revenir d'une page

A.2 Fichiers et dossiers

Chemin	Contenu
`~/.config/asterism/` ASTERISM `~/.config/omarchy/ai-agents-map/` OMARCHY	Réglages, la liste des serveurs MCP, les agents recherchés, les connexions échouées.
`~/.local/state/ai-agents-map/backups/`	Les sauvegardes de chaque fichier modifié par l'app.
`~/.local/state/ai-agents-map/jobs.json`	Les téléchargements et installations interrompus.
`~/.local/state/ai-agents-map/reorganize.json`	Le journal du dernier rangement, pour Annuler.
`~/.cache/ai-agents-map/`	Résultats de recherche, listes d'outils, vérifications de mises à jour, icônes des modules, le dernier Smart Check (`smart-check.json`) et les mises à jour MCP des vérifications IA (`mcp-updates.json`).
`~/.local/state/omarchy-plugin-ai/runs.json`	Les recherches et tâches IA de cette app et d'autres plugins, avec leurs jetons : les statistiques de Smart Check.
`~/.config/systemd/user/ai-agents-map-providers.service`	Le veilleur qui démarre les LLM locaux avec leurs agents.

A.3 Glossaire

Approbation	Un agent demande avant de modifier un fichier ou d'exécuter une commande (voir Les mots expliqués).
Contexte	Combien de jetons de texte un modèle peut garder en tête à la fois.
GGUF	Le format de fichier de modèles utilisé par llama.cpp, LM Studio, Ollama et KoboldCpp.
Injection de prompt	Des instructions cachées dans une page, un fichier ou la réponse d'un outil qui essaient d'orienter un agent.
Quantification	Stocker un modèle avec moins de bits par poids (par exemple *Q4_K_M*, environ 4,8 bits) pour qu'il soit plus petit, avec une faible perte de qualité.
Mélange d'experts	Un modèle où seule une partie des paramètres travaille pour chaque jeton (*35B-A3B* : 35 milliards au total, 3 milliards actifs), ce qui le rend rapide pour sa taille.
Sandbox	Un espace isolé où tournent les commandes d'un agent, qui n'atteignent que le dossier du projet et ce qui est autorisé.
Jeton	Un morceau de texte, environ les trois quarts d'un mot, que les modèles lisent et écrivent.
Service utilisateur	Un programme d'arrière-plan que systemd démarre pour ton utilisateur seulement, sans droits d'administrateur.